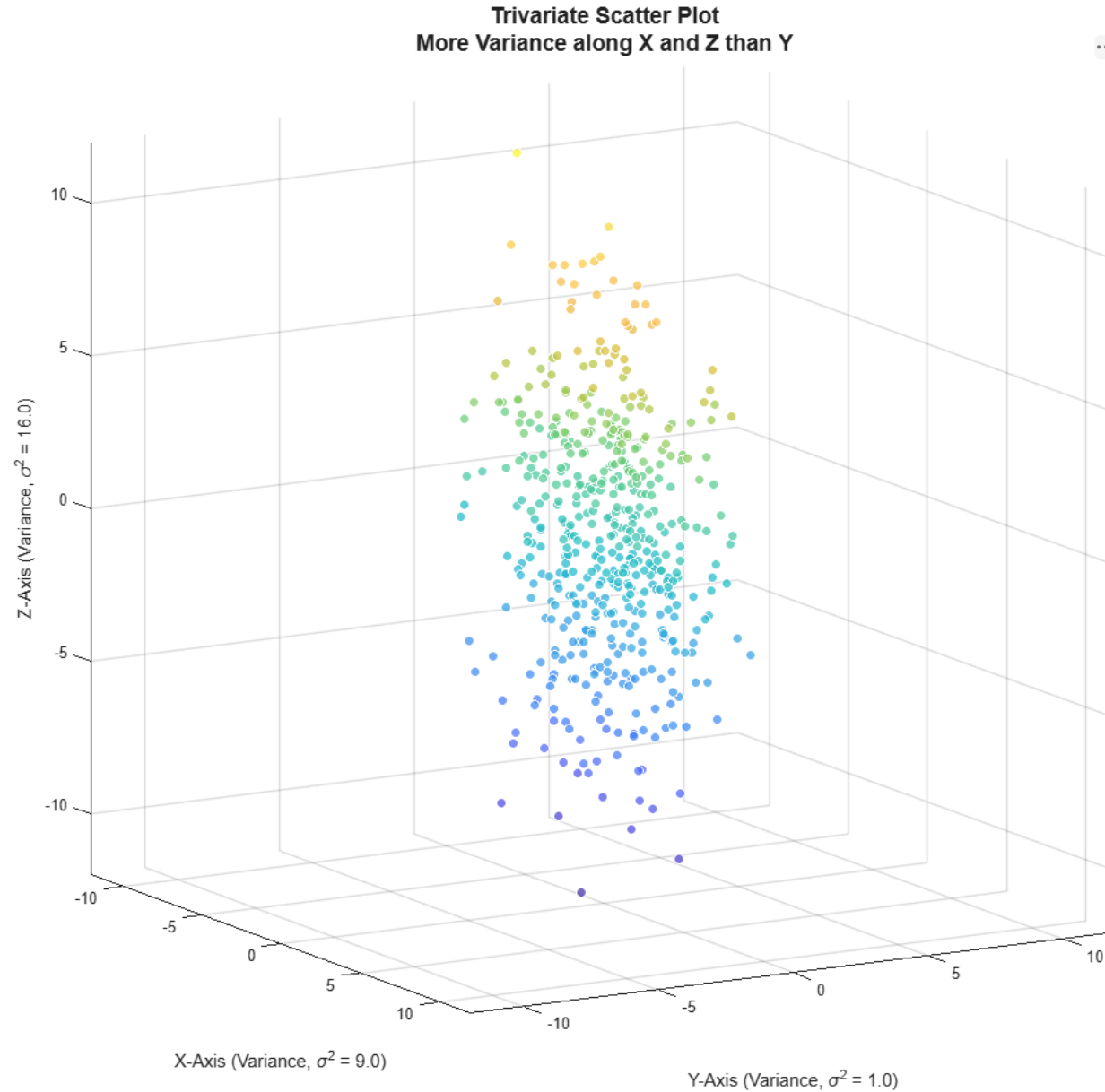


Statistical Distance

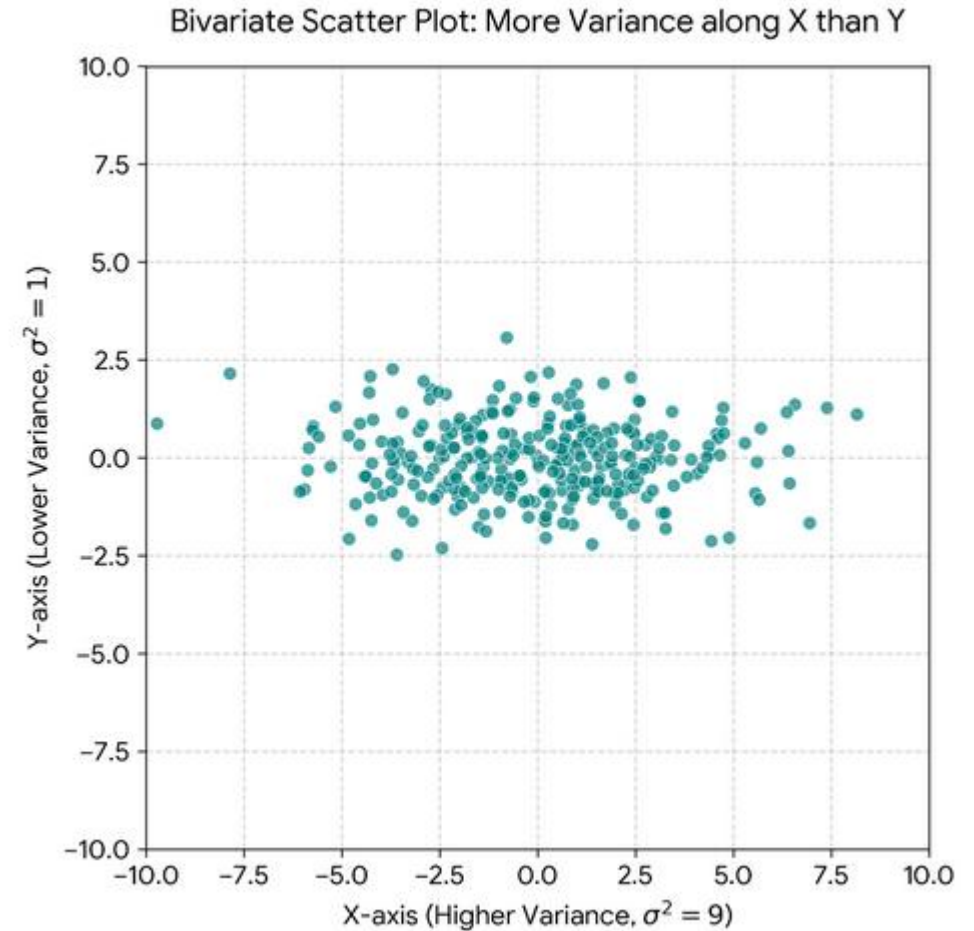


Reference Books

- ❖ Brereton R. J., The Chi Squared and Multinormal Distributions, J. Chemometrics 2015, 29: 9–12
- ❖ Johnson R A and D W Wichern, Applied Multivariate Statistical Analysis, Prentice Hall, 2002.
- ❖ Maiti J, Multivariate Statistical Modeling in Engineering and Management, CRC Press, 2023

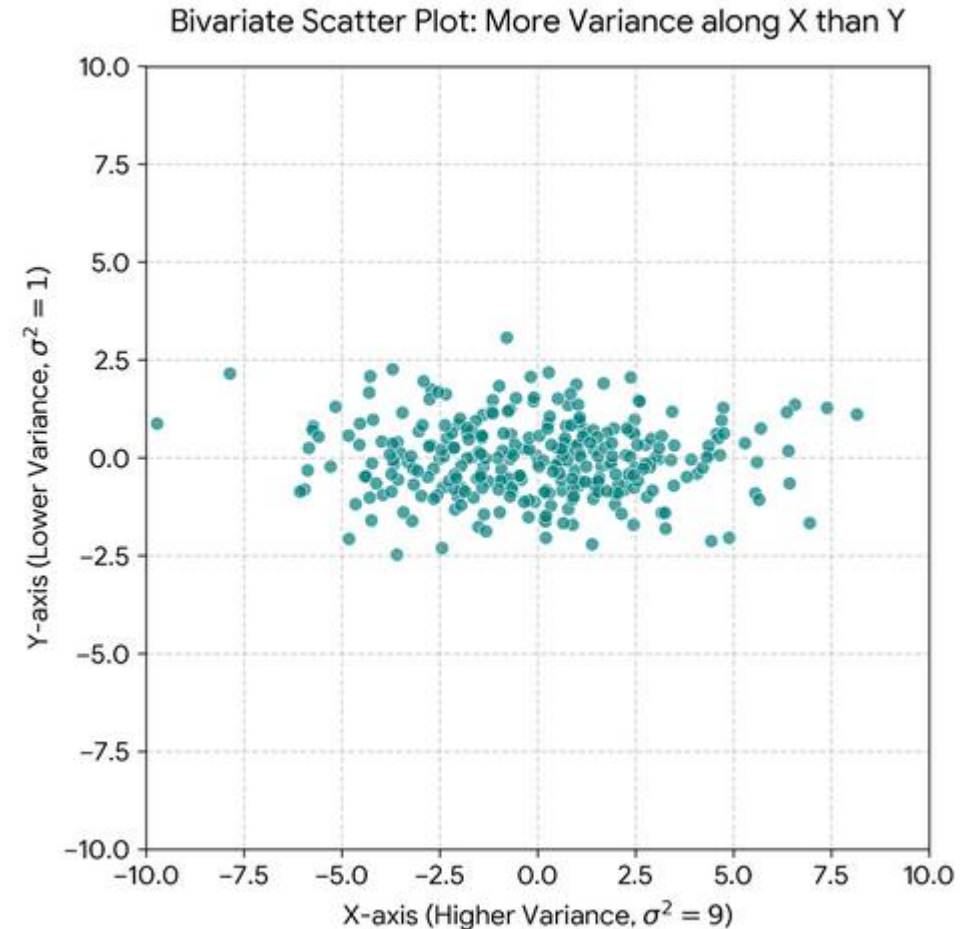
Two-Dimensional Scattered Data Plot

- ❖ Consider Fig. on RHS, a bivariate scatter plot of variables X and Y
- ❖ The population (sample) is characterized by the two variables X and Y
- ❖ The population (sample) shows scattered values along both X and Y dimensions.
- ❖ Variability along X (σ_X^2) is greater than along Y (say σ_Y^2),
- ❖ The two variables are not correlated



Two-Dimensional Scattered Data Plot

- ❖ **Data Generation:** The dataset consists of 300 points drawn from a bivariate normal distribution where the dimensions are independent.
- ❖ **X-axis Variance:** The standard deviation along the X-axis is set to $\sigma_x=3.0$, yielding a variance of $\sigma_x^2=9.0$
- ❖ **Y-axis Variance:** The standard deviation along the Y-axis is set to $\sigma_y=1.0$, yielding a variance of $\sigma_y^2=1.0$
- ❖ **Visual Scale:** To accurately represent the true geometric difference in variance without stretching distortion, the plot utilizes an **equal aspect ratio** with symmetric axis limits ranging from -10 to 10 .
- ❖ The data points stretch horizontally into an elongated oval shape, clearly demonstrating that the spread (dispersion) of data is 3 times wider along the X-axis than along the Y-axis.



Python Code: 2 D Scatter Plot

```
import numpy as np
import matplotlib.pyplot as plt

# Set random seed for reproducibility
np.random.seed(42)

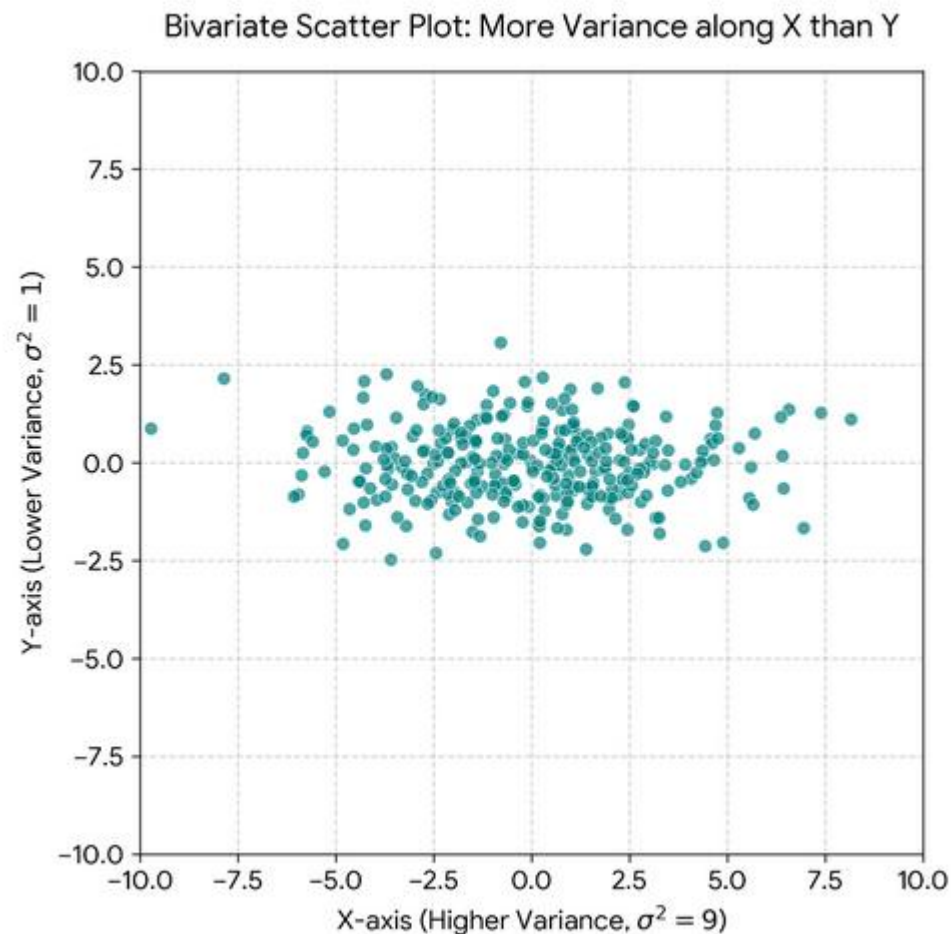
# Generate data: X has a variance of 9, Y has a variance of 1
x = np.random.normal(loc=0, scale=3.0, size=300)
y = np.random.normal(loc=0, scale=1.0, size=300)

# Create the scatter plot using subplots
fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(x, y, alpha=0.7, color='teal', edgecolor='w', linewidth=0.5)

# Set titles and labels using LaTeX formatting
ax.set_title('Bivariate Scatter Plot: More Variance along X than Y',
            fontsize=14, pad=15)
ax.set_xlabel('X-axis (Higher Variance,  $\sigma^2 = 9$ )', fontsize=12)
ax.set_ylabel('Y-axis (Lower Variance,  $\sigma^2 = 1$ )', fontsize=12)

# Set equal aspect ratio and matching limits to perfectly visualize the
variance difference
ax.set_aspect('equal', adjustable='box')
ax.set_xlim(-10, 10)
ax.set_ylim(-10, 10)
ax.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.savefig('scatter_plot_variance.png', dpi=300)
```



Matlab Code: 2 D Scatter Plot

```
% 1. Set seed for reproducibility
rng(42);

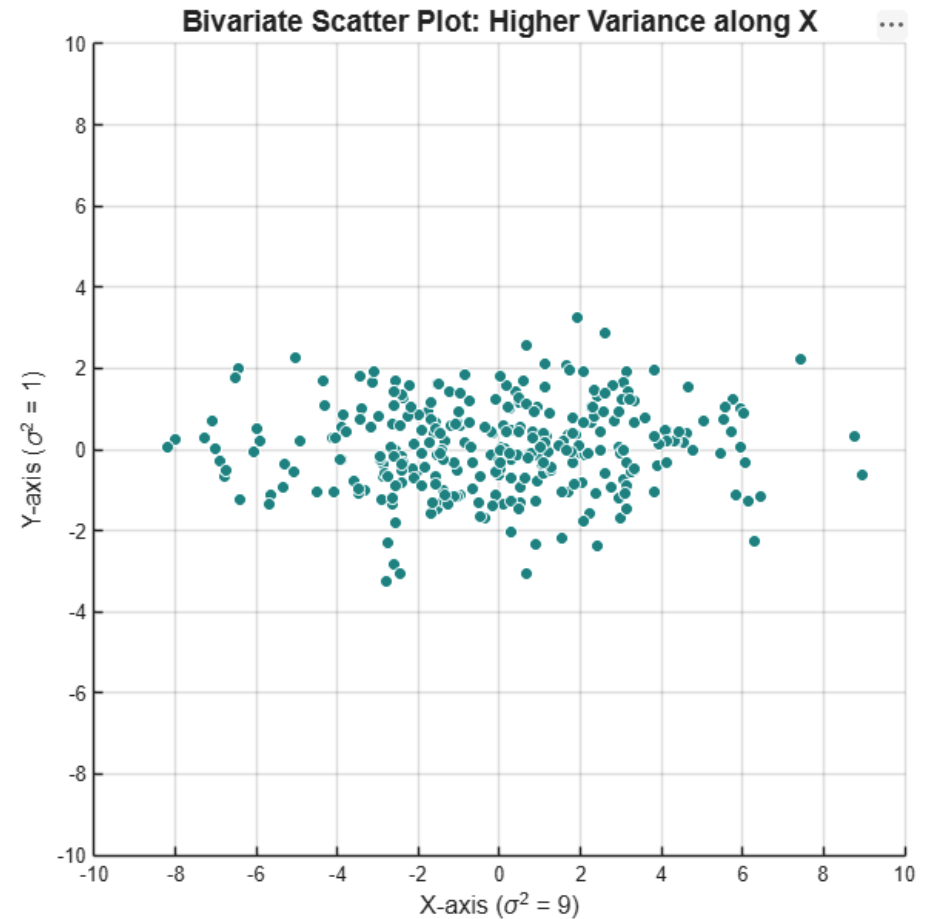
% 2. Define parameters (X has higher variance than Y)
num_points = 300;
mean_x = 0; mean_y = 0;
std_x = 3.0; std_y = 1.0; % Variance X = 9, Variance Y = 1

% 3. Generate independent normally distributed data
x = mean_x + std_x * randn(num_points, 1);
y = mean_y + std_y * randn(num_points, 1);

% 4. Create the scatter plot
figure('Color', 'w'); % Open a white background figure
scatter(x, y, 35, 'filled', 'MarkerFaceColor', [0 0.5 0.5], 'MarkerEdgeColor', 'w');

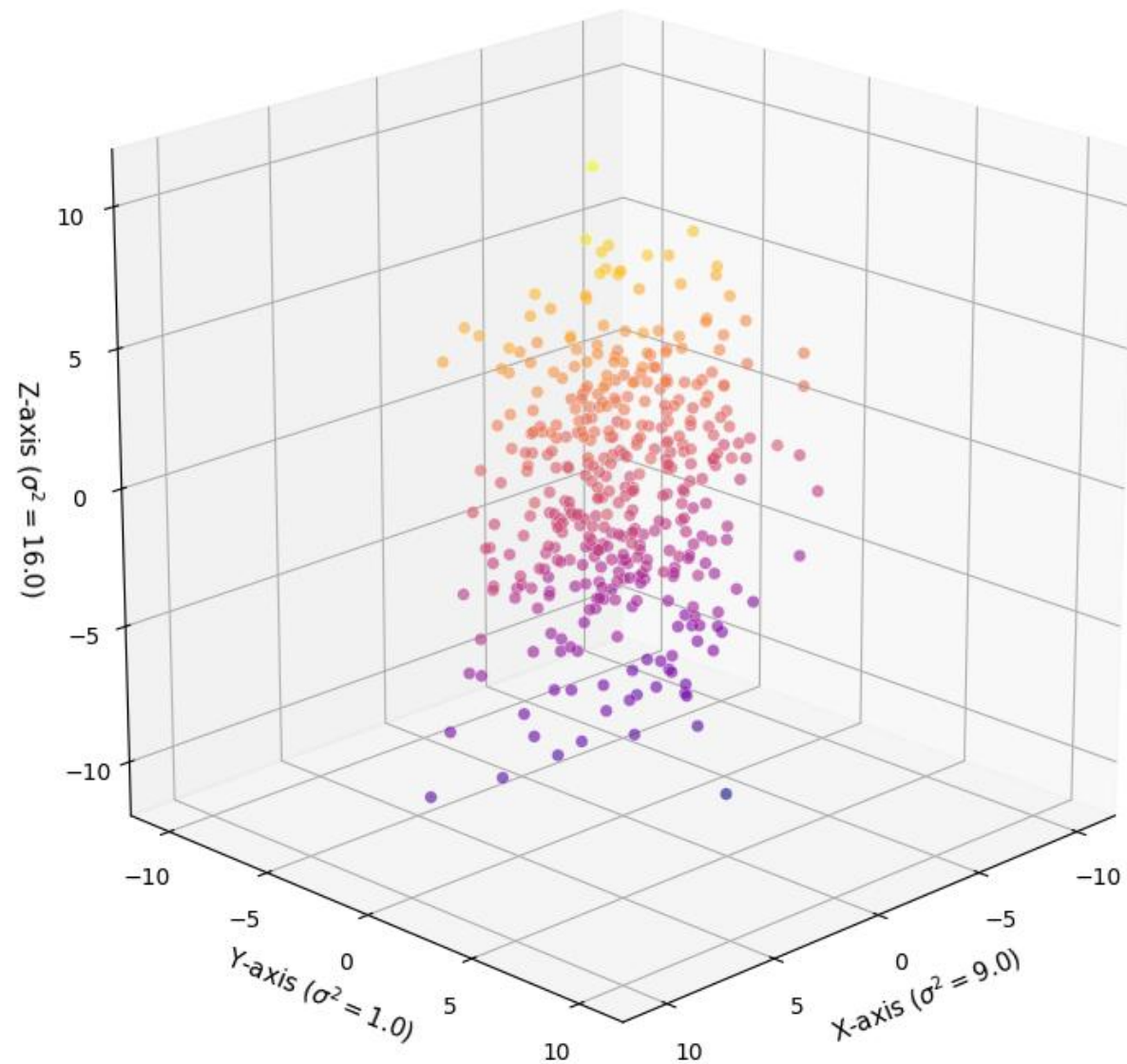
% 5. Format labels and titles
title('Bivariate Scatter Plot: Higher Variance along X', 'FontSize', 14);
xlabel('X-axis ( $\sigma^2 = 9$ )', 'FontSize', 12);
ylabel('Y-axis ( $\sigma^2 = 1$ )', 'FontSize', 12);

% 6. Adjust axes for true geometric representation
grid on;
axis equal; % CRITICAL: Keeps 1 unit on X equal to 1 unit on Y
xlim([-10, 10]);
ylim([-10, 10]);
```



Python Code: 3 D Scatter Plot

```
import numpy as np
# Set random seed for reproducibility
np.random.seed(101)
# Generate 3D data with specified variances
# X variance: std = 3.0 -> var = 9.0
# Y variance: std = 1.0 -> var = 1.0
# Z variance: std = 4.0 -> var = 16.0
n_samples = 400
x = np.random.normal(loc=0, scale=3.0, size=n_samples)
y = np.random.normal(loc=0, scale=1.0, size=n_samples)
z = np.random.normal(loc=0, scale=4.0, size=n_samples)
# Create 3D subplot
fig, ax = plt.subplots(figsize=(8, 8), subplot_kw={"projection": "3d"})
# Plot the 3D scatter points
sc = ax.scatter(x, y, z, c=z, cmap='plasma', alpha=0.6, edgecolors='w', linewidth=0.3, s=30)
# Set labels using LaTeX formatting
ax.set_xlabel('X-axis ( $\sigma^2 = 9.0$ )', fontsize=11)
ax.set_ylabel('Y-axis ( $\sigma^2 = 1.0$ )', fontsize=11)
ax.set_zlabel('Z-axis ( $\sigma^2 = 16.0$ )', fontsize=11)
ax.set_title('Tri-variate Scatter Plot (Uniform 1:1:1 Spatial Axes)', fontsize=13, pad=20)
# CRITICAL: Force the physical bounding box to be a perfect cube
ax.set_box_aspect([1, 1, 1])
# Enforce uniform coordinate limits across all axes
ax.set_xlim(-12, 12)
ax.set_ylim(-12, 12)
ax.set_zlim(-12, 12)
# Adjust viewing angle for an optimal 3D perspective
ax.view_init(elev=20, azim=45)
plt.tight_layout()
plt.show()
```



Matlab Code: 3-D Scatter Plot

```
% 1. Set random seed for reproducibility
rng(101);

% 2. Define statistical parameters (Standard Deviations)
num_points = 500;
std_x = 3.0; % Variance (sigma^2) = 9.0
std_y = 1.0; % Variance (sigma^2) = 1.0
std_z = 4.0; % Variance (sigma^2) = 16.0

% 3. Generate independent normally distributed data
x = std_x * randn(num_points, 1);
y = std_y * randn(num_points, 1);
z = std_z * randn(num_points, 1);

% 4. Create a clean figure with a white background
figure('Color', 'w', 'Position', [100, 100, 750, 650]);

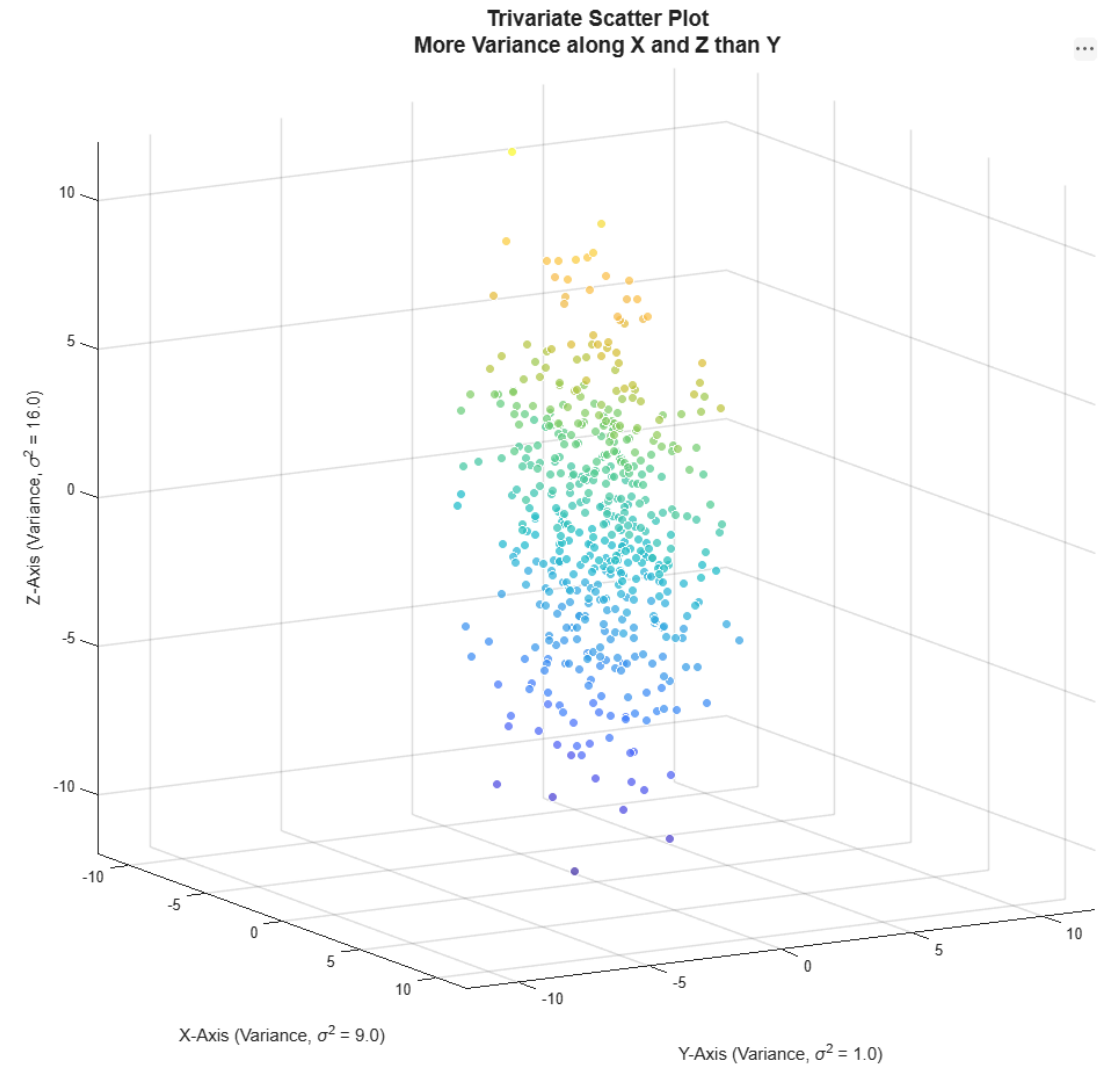
% 5. Generate 3D scatter plot
% Points are color-mapped by their Z-axis values using the 'parula' colormap
scatter3(x, y, z, 35, z, 'filled', 'MarkerEdgeColor', 'w', 'MarkerFaceAlpha', 0.7);
colormap('parula');
cb = colorbar;
ylabel(cb, 'Z-Value Intensity', 'FontSize', 10);

% 6. Add labels containing the explicit variance (\sigma^2) values
title({'Trivariate Scatter Plot', 'More Variance along X and Z than Y'}, 'FontSize', 14, 'FontWeight', 'bold');
xlabel('X-Axis (Variance, \sigma^2 = 9.0)', 'FontSize', 11);
ylabel('Y-Axis (Variance, \sigma^2 = 1.0)', 'FontSize', 11);
zlabel('Z-Axis (Variance, \sigma^2 = 16.0)', 'FontSize', 11);

% 7. CRITICAL: Force uniform spatial distance scale along all 3 axes
grid on;
axis equal; % Ensures 1 unit physically looks the same on X, Y, and Z

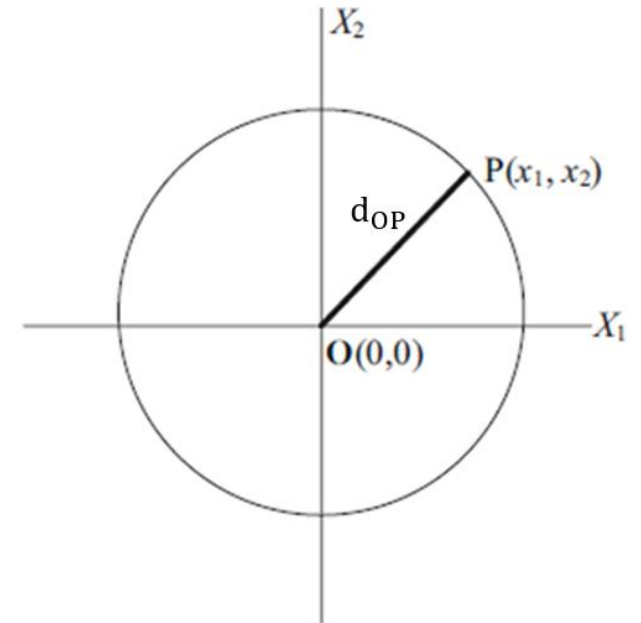
% 8. Enforce identical bounding limits to maintain a perfect cubic grid
axis_limit = 12;
xlim([-axis_limit, axis_limit]);
ylim([-axis_limit, axis_limit]);
zlim([-axis_limit, axis_limit]);

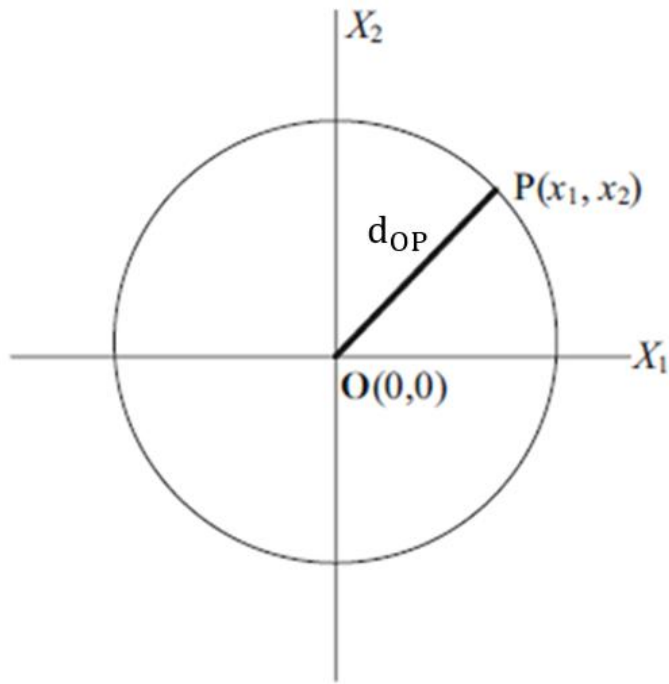
% 9. Set the optimal 3D isometric perspective view (Azimuth, Elevation)
view(40, 25);
```



Some Essential Background

- ❖ **Statistical Distance:** Crucial to multivariate data analysis.
- ❖ Let us first learn about **Euclidian Distance** which may be familiar to you
and
- ❖ then get into the meaning of **Statistical Distance**.
- ❖ Euclidean Distance is a measure of the distance between two points in a **Cartesian coordinate system**.
- ❖ Denoted by X_1 and X_2 axes, the Cartesian coordinate system can be represented (see Fig.)
- ❖ The distance between the origin $O (0,0)$ and a point $P (x_1, x_2)$ may be written as: $d(OP) = \sqrt{x_1^2 + x_2^2}$





- ❖ The specific algebraic formula used to calculate this distance was not actually written by Euclid.
- ❖ Instead, the distance formula itself was first published in 1731 by the French mathematician **Alexis Clairaut**.
- ❖ It builds upon the Pythagorean theorem and the Cartesian coordinate system introduced by René Descartes in 1637



Alexis Clairaut

$$d_{OP}^2 = x_1^2 + x_2^2$$

d_{OP} = radius of the circle

Euclidean Distance: General Form

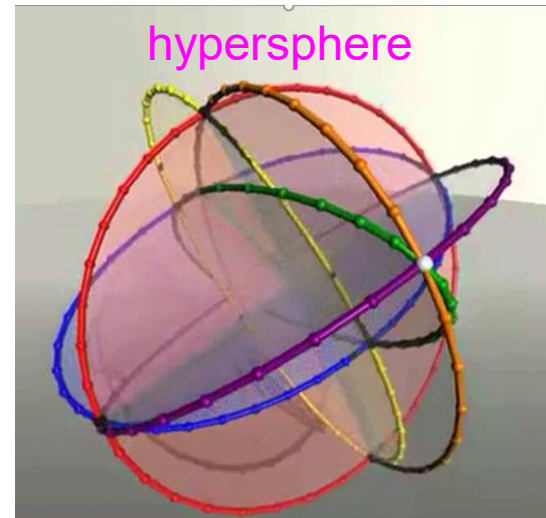
❖ Measure the distance from a fixed-point Q with coordinate $(q_1, q_2, \dots, q_p)$ to

❖ P with coordinates $(x_1, x_2, \dots, x_p)$, the distance $d(PQ)$ is

$$d(PQ) = \sqrt{(x_1 - q_1)^2 + (x_2 - q_2)^2 + \dots + (x_p - q_p)^2}$$

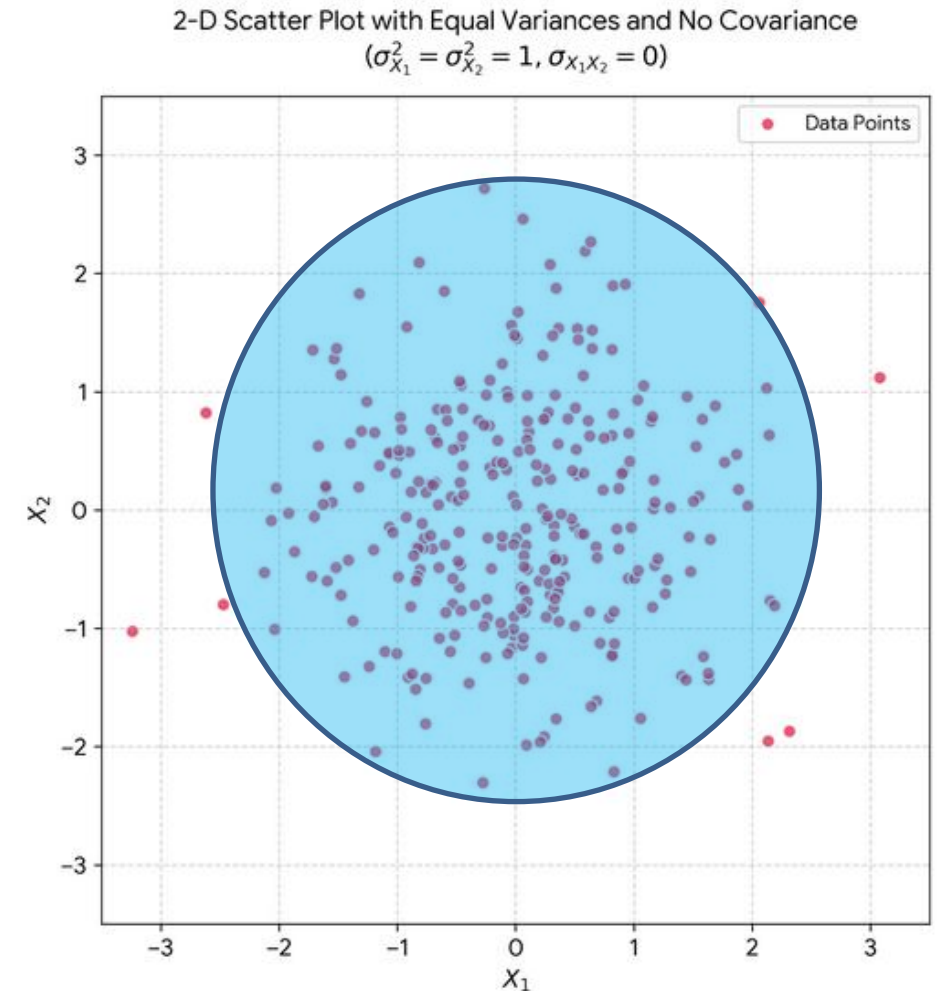
❖ Equation is that of a hypersphere centered at some arbitrary point Q, respectively.

❖ All points on the surface of this **hypersphere** are equidistance (d) from its center.



A. Uncorrelated X_1 and X_2 With Equal Variances

- ❖ Let us now consider four applications A, B, C, and D, each characterized with two variables X_1 and X_2 .
- ❖ The scatter plots for each of the four applications are shown in this and other slides
- ❖ Every point on the scatter is a bivariate observation
- ❖ The data for process A exhibit a circular pattern that is indicative of **equal spread (variance)** for the two variables X_1 and X_2 .
- ❖ In such a case, there is **no correlation** between the two variables X_1 and X_2 .

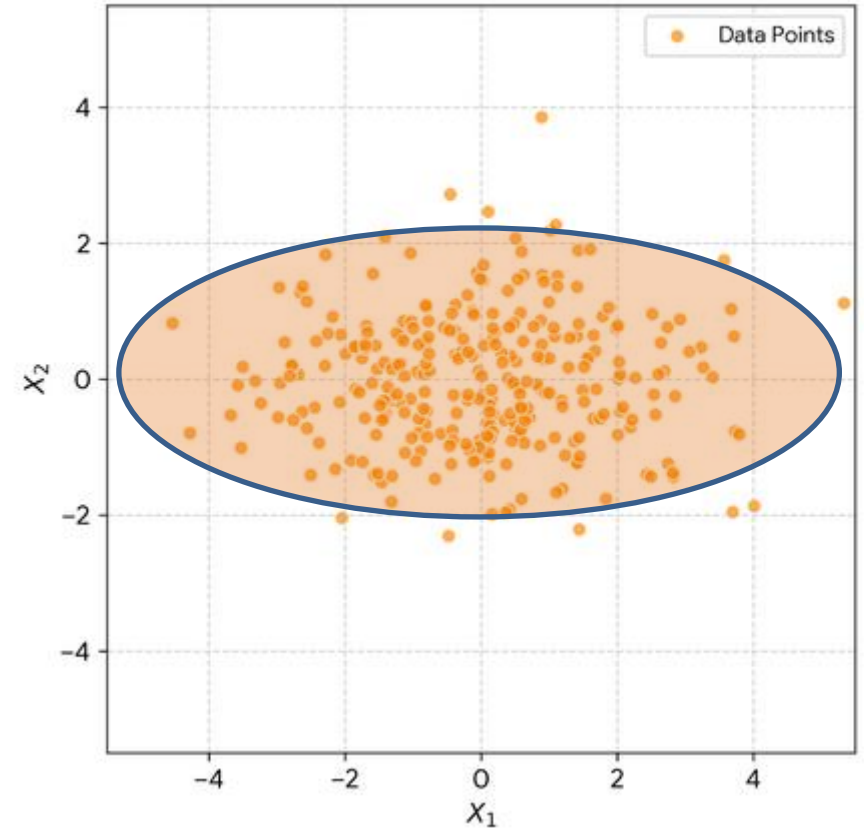


$$\mathbf{\Sigma} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

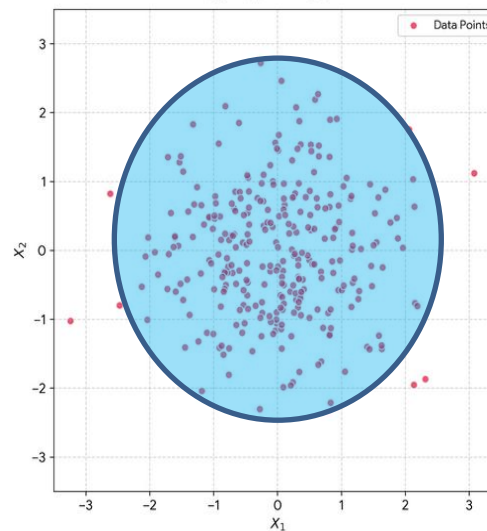
B. Uncorrelated X_1 and X_2 with Different Variances

- ❖ The data for process A exhibit an elliptic pattern that is indicative of **different spread (variance)** for the two variables X_1 and X_2 .
- ❖ However, there is **no correlation** between the two variables X_1 and X_2 .

2-D Scatter Plot with Unequal Variances and No Covariance
($\sigma_{X_1}^2 = 3, \sigma_{X_2}^2 = 1, \sigma_{X_1 X_2} = 0$)



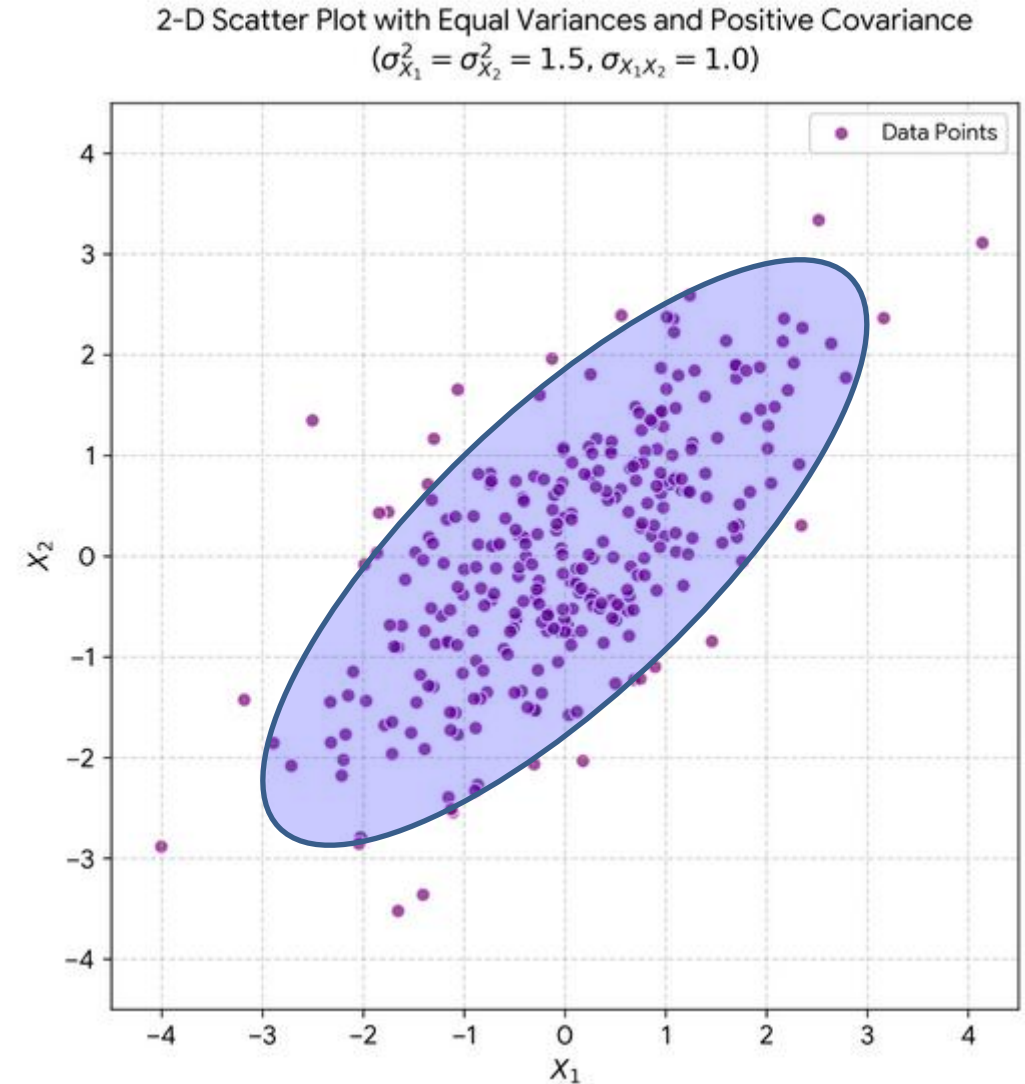
2-D Scatter Plot with Equal Variances and No Covariance
($\sigma_{X_1}^2 = \sigma_{X_2}^2 = 1, \sigma_{X_1 X_2} = 0$)



$$\mathbf{\Sigma} = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$$

C. Positively Correlated X_1 and X_2 with Same Variances

- ❖ The data for process C exhibit an elliptic pattern with **equal variance** for the two variables X_1 and X_2 .
- ❖ However, there is **a positive correlation** between the two variables X_1 and X_2 .



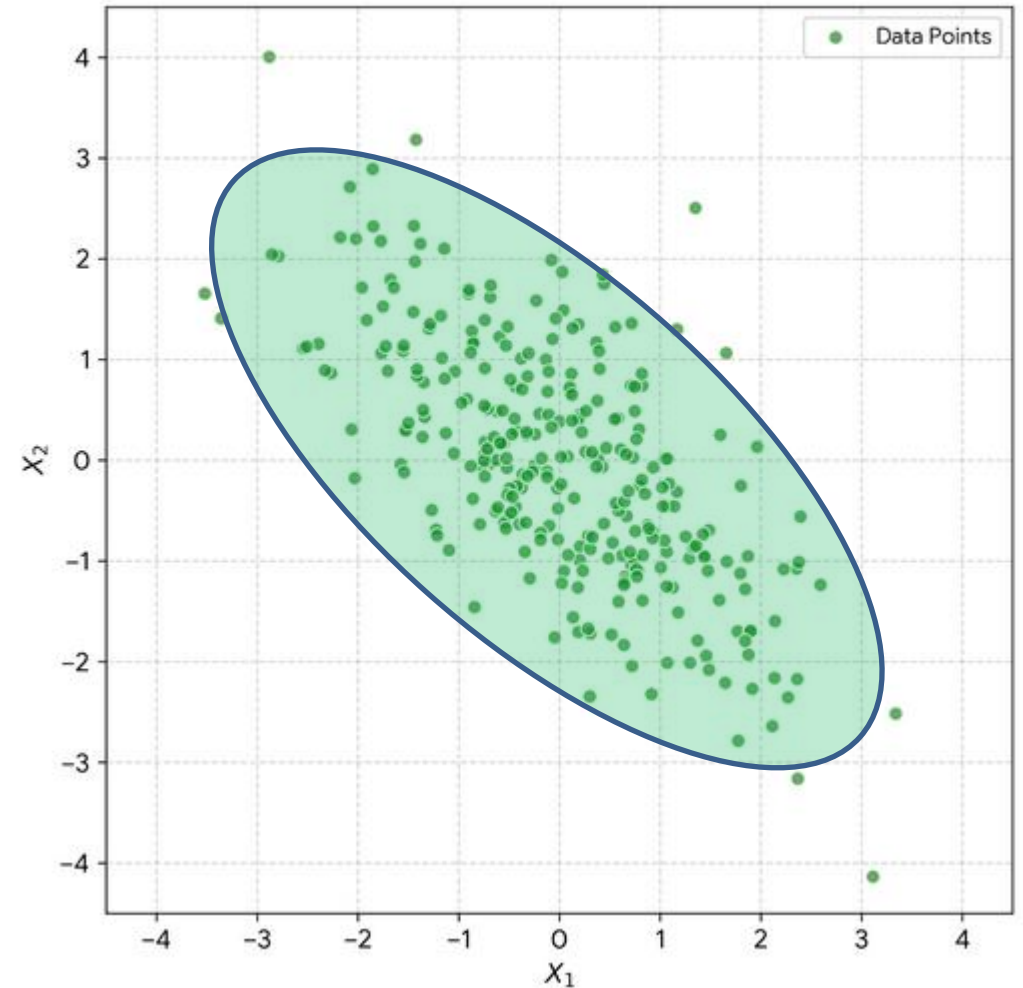
$$\mathbf{\Sigma} = \begin{bmatrix} 1.5 & 1 \\ 1 & 1.5 \end{bmatrix}$$

C. Negatively Correlated X_1 and X_2 with Same Variances

2-D Scatter Plot with Equal Variances and Negative Covariance

$$(\sigma_{X_1}^2 = \sigma_{X_2}^2 = 1.5, \sigma_{X_1 X_2} = -1.0)$$

- ❖ The data for process C exhibit an elliptic pattern with **equal variances** for the two variables X_1 and X_2 .
- ❖ However, there is **a negative correlation** between the two variables X_1 and X_2 .

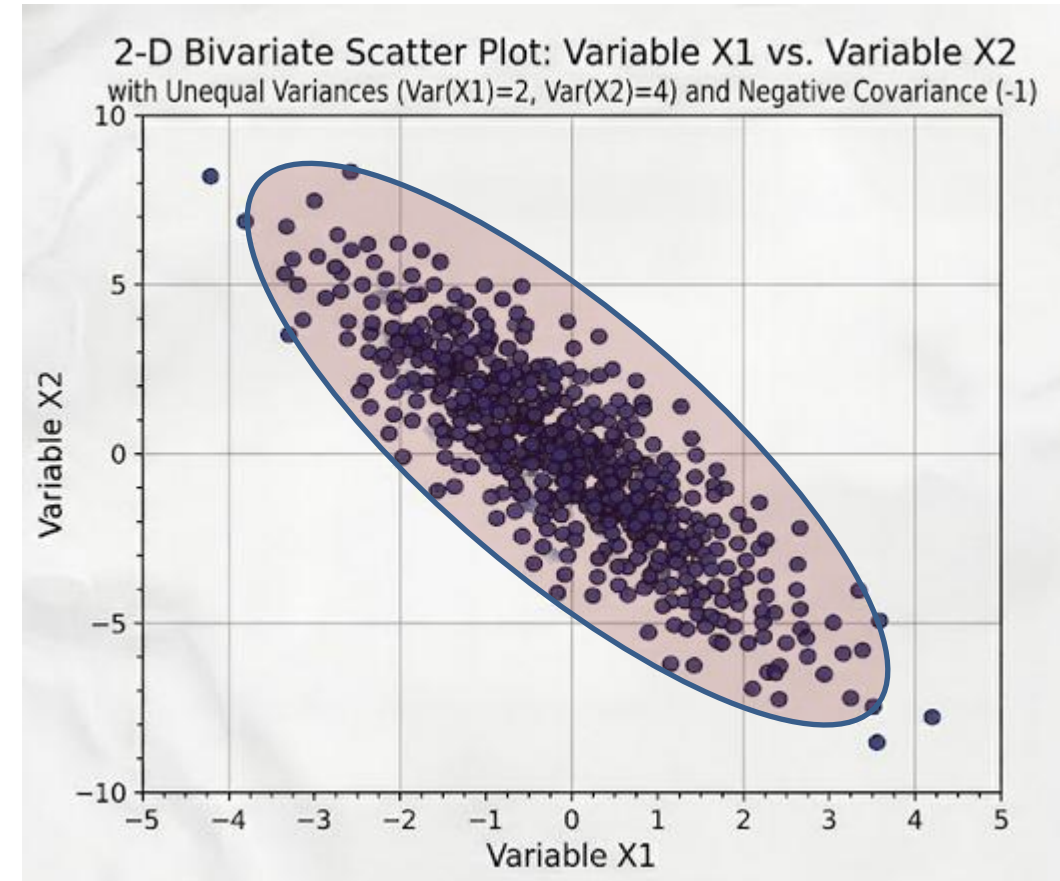


$$\Xi = \begin{bmatrix} 1.5 & -1 \\ -1 & 1.5 \end{bmatrix}$$

D. Negatively Correlated X_1 and X_2 with Different Variances

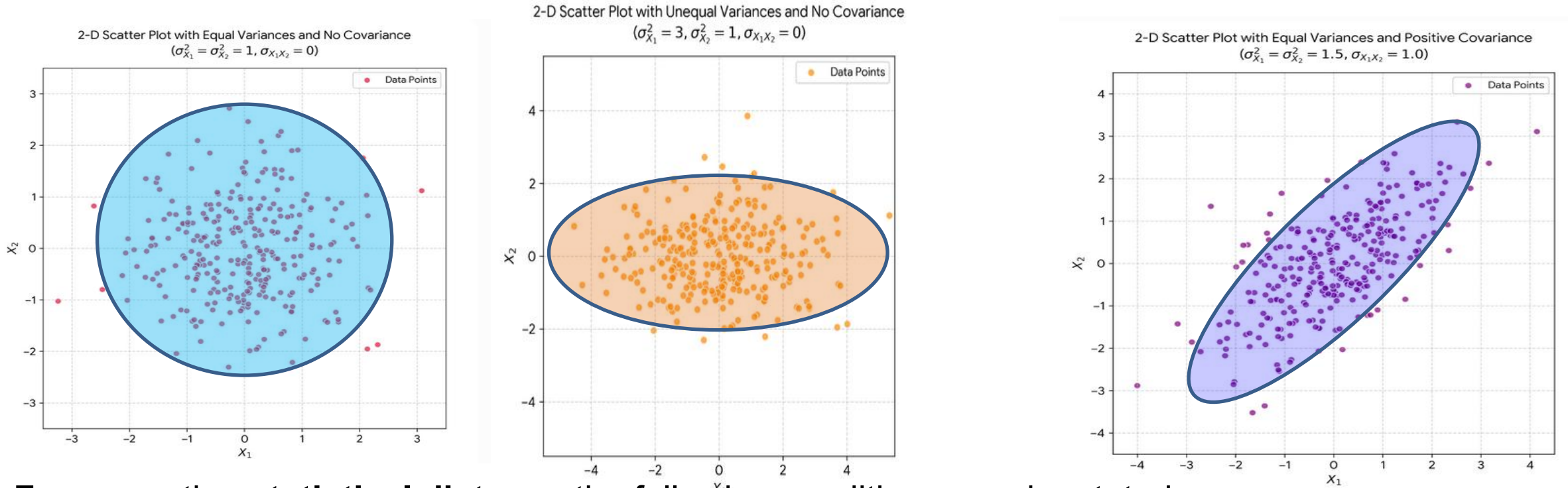
❖ The data for process C exhibit an elliptic pattern that is indicative of **unequal variances** for the two variables X_1 and X_2 .

❖ Moreover, there is **a negative correlation** between the two variables X_1 and X_2 .



$$\Xi = \begin{bmatrix} 2 & -1 \\ -1 & 4 \end{bmatrix}$$

D. Preamble for Statistical Distance

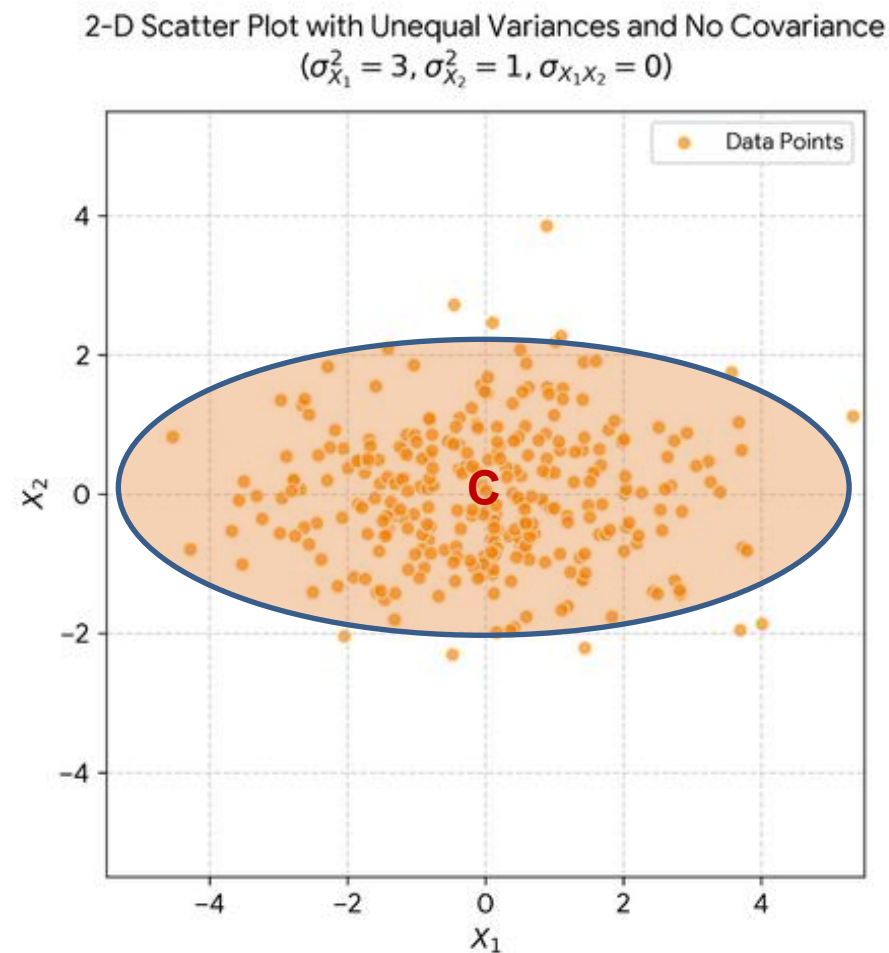


For computing **statistical distance** the following conditions may be stated:

- ❖ Every application is characterized by two (bivariate) or more variables >2 (multivariate) system
- ❖ The variables, each taken as an individual group, may or may not have equal variance.
- ❖ The variables, may be either independent or correlated
- ❖ Variance- covariance relationships given by the Σ matrix define the pattern in the data.
- ❖ **The pattern may be captured through appropriate distance measure.**

Some Essential Background

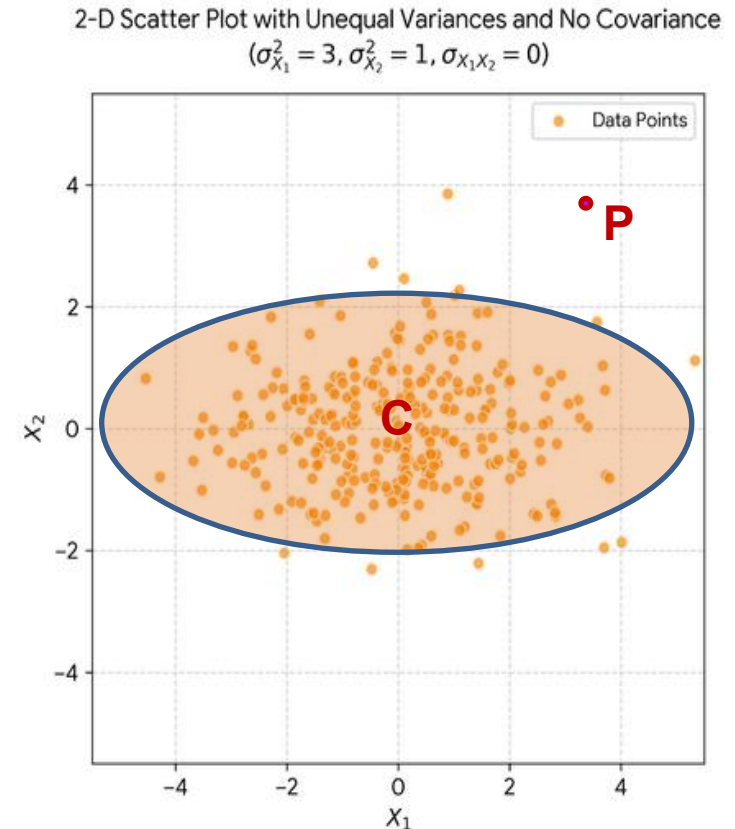
- ❖ Now let us consider Application B.
- ❖ The pattern that defines the data is an ellipse.
- ❖ Here the major and minor axes of the ellipse are **parallel** to the XY-coordinate system.
- ❖ The center **C** is at the origin (0,0),



Uncorrelated Variables

$$\Sigma = \begin{bmatrix} \sigma_{11} & 0 \\ 0 & \sigma_{22} \end{bmatrix}$$

- ❖ No correlation between X_1 and X_2
- ❖ X_1 and X_2 are independent
- ❖ From Process C, it is seen that σ_{11} is 3 times more than σ_{22} .
- ❖ To obtain the **statistical distance** of $P(X_1, X_2)$ from $C(0,0)$, we first require the coordinates **to be on equal footing (standardizing)**.
- ❖ Standardizing is possible by the following transformation



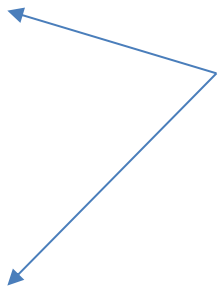
Equal Footing

❖ We defined dimensionless coordinates as given below

$$x_1^* = \frac{x_1 - 0}{\sqrt{\sigma_{11}}}$$

$$x_2^* = \frac{x_2 - 0}{\sqrt{\sigma_{22}}}$$

Note the square root on the variance to give the standard deviation



❖ Now, we will **define** the **statistical distance** from the centre (here origin) in terms of the transformed variables as

$$\frac{x_1^2}{\sigma_{11}} + \frac{x_2^2}{\sigma_{22}} = d^2$$

Equal Footing

- ❖ Now, we will define the distance in terms of the transformed variables as

$$\frac{x_1^{*2}}{\sigma_{11}} + \frac{x_2^{*2}}{\sigma_{22}} = d^2$$

- ❖ Eqn. above may be related to the equation of an ellipse, centered at the origin (0, 0)

- ❖ We may also write as follows

$$x_1^{*2} + x_2^{*2} = d^2$$

= ED $\sigma_{11} = \sigma_{22}$ Centre (0,0)
Not Euclidean distance (ED)

- ❖ Now, let us do some elementary matrix manipulation with x and σ s as shown below.

Most general

$$\Sigma = \begin{bmatrix} \sigma_{11} & 0 \\ 0 & \sigma_{22} \end{bmatrix}$$

$$\Sigma^{-1} = \begin{bmatrix} \frac{1}{\sigma_{11}} & 0 \\ 0 & \frac{1}{\sigma_{22}} \end{bmatrix}$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

$$\mathbf{x}' = [x_1 \ x_2]$$

$p = 2$
more # of variables

Even More General
 $\begin{bmatrix} \sigma_{11} & \sigma_{12} \\ \sigma_{12} & \sigma_{22} \end{bmatrix}$
 More general expression
 Most general expression

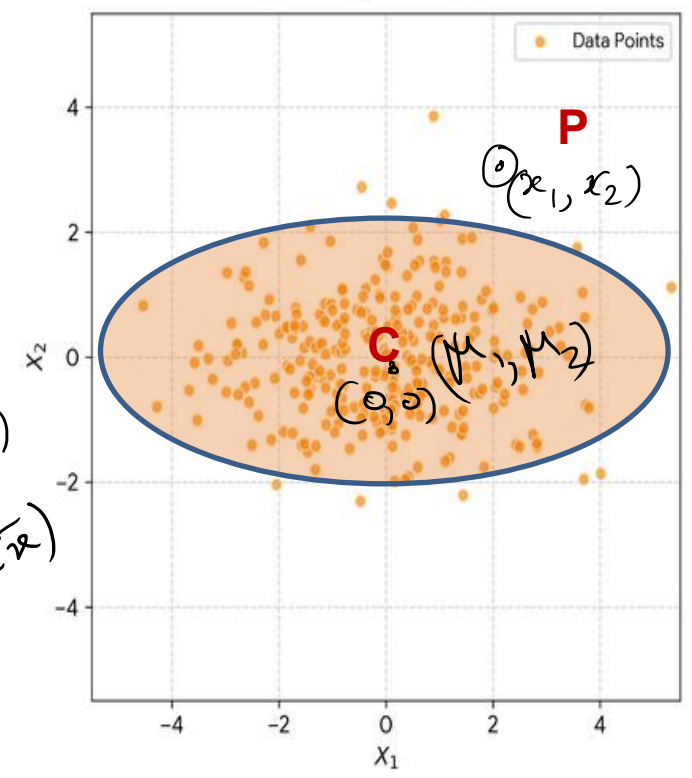
$$\mathbf{x}' \Sigma^{-1} \mathbf{x}$$

$$= [x_1 \ x_2] \begin{bmatrix} \frac{1}{\sigma_{11}} & 0 \\ 0 & \frac{1}{\sigma_{22}} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$\mathbf{x}' \Sigma^{-1} \mathbf{x} = \frac{x_1^2}{\sigma_{11}} + \frac{x_2^2}{\sigma_{22}}$$

$$\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 = s^2$$

2-D Scatter Plot with Unequal Variances and No Covariance
 $(\sigma_{x_1}^2 = 3, \sigma_{x_2}^2 = 1, \sigma_{x_1 x_2} = 0)$



$$(X'X) = \begin{bmatrix} 12 & 0 & 0 \\ 0 & 8 & 0 \\ 0 & 0 & 12 \end{bmatrix}$$

$$(X'X)^{-1} = \begin{bmatrix} 1/12 & 0 & 0 \\ 0 & 1/8 & 0 \\ 0 & 0 & 1/12 \end{bmatrix}$$

linear regression

Definition of Statistical Distance

$$\Sigma = \begin{bmatrix} \sigma_{11} & 0 \\ 0 & \sigma_{22} \end{bmatrix}$$

$$\Sigma^{-1} = \begin{bmatrix} \frac{1}{\sigma_{11}} & 0 \\ 0 & \frac{1}{\sigma_{22}} \end{bmatrix} \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad \mathbf{x}' = [x_1 \ x_2]$$

$$\mathbf{x}' \Sigma^{-1} \mathbf{x} = [x_1 \ x_2] \begin{bmatrix} \frac{1}{\sigma_{11}} & 0 \\ 0 & \frac{1}{\sigma_{22}} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

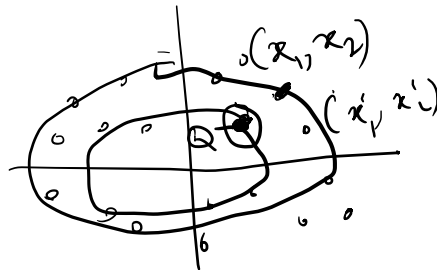
$$\mathbf{x}' \Sigma^{-1} \mathbf{x} = \frac{x_1^2}{\sigma_{11}} + \frac{x_2^2}{\sigma_{22}}$$

$$\frac{x_1^2}{\sigma_{11}} + \frac{x_2^2}{\sigma_{22}} = d^2$$

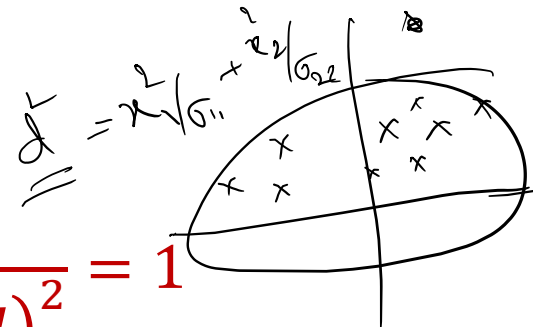
$$\mathbf{x}' \Sigma^{-1} \mathbf{x} = d^2$$

$$x_1^{*2} + x_2^{*2} = d^2$$

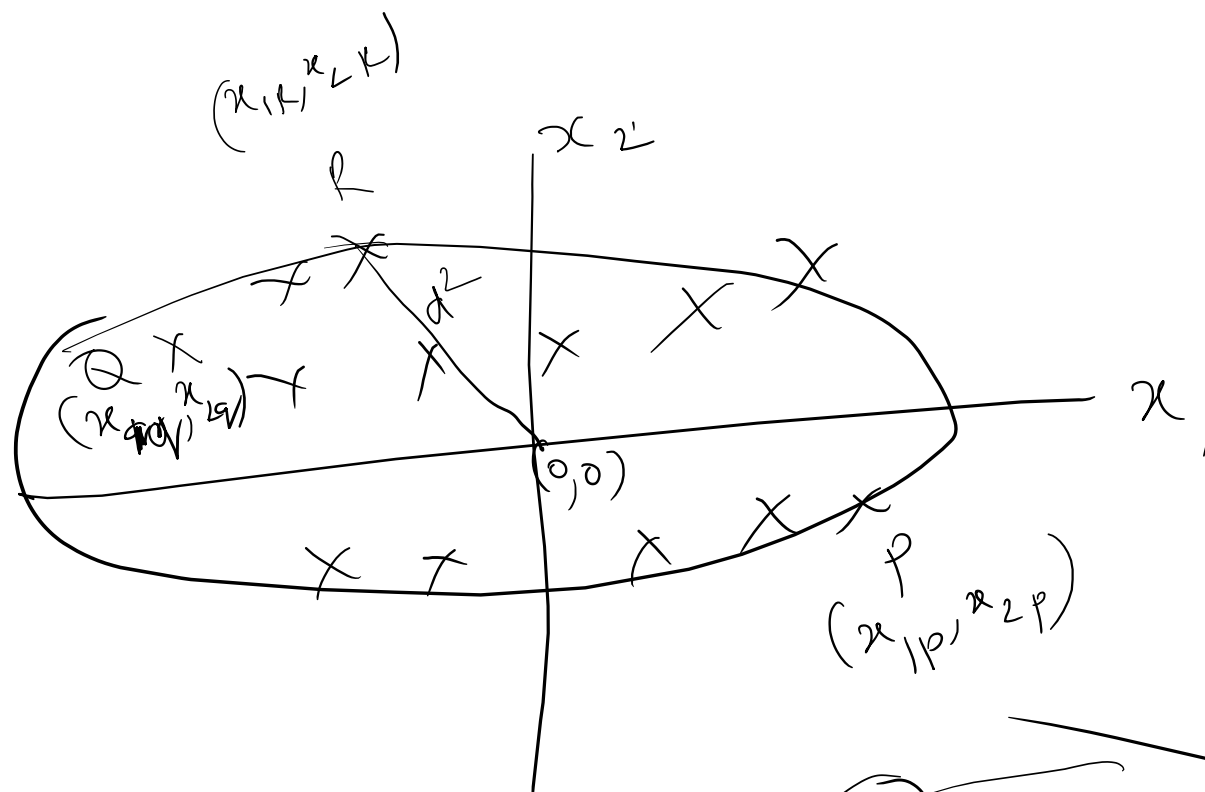
So we define the statistical distance in terms of coordinate location of **a given point** $\mathbf{x}$ identified by (x_1, x_2) as well as the standard deviations associated with the standard deviations along X_1 and X_2



$$\frac{x_1^2}{(\sqrt{\sigma_{11}}d)^2} + \frac{x_2^2}{(\sqrt{\sigma_{22}}d)^2} = 1$$

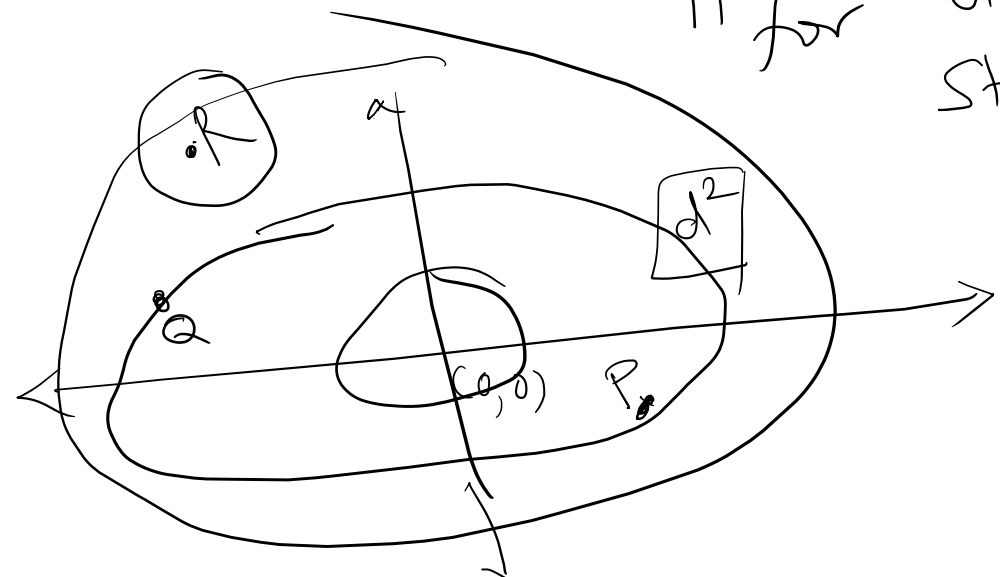


Locus of points governed by the equation results in **an ellipse**



$$\frac{x_{1R}^2}{\sigma_{11}} + \frac{x_{2R}^2}{\sigma_{22}} = d_R^2$$

appt. value for d^2
Statistical distance

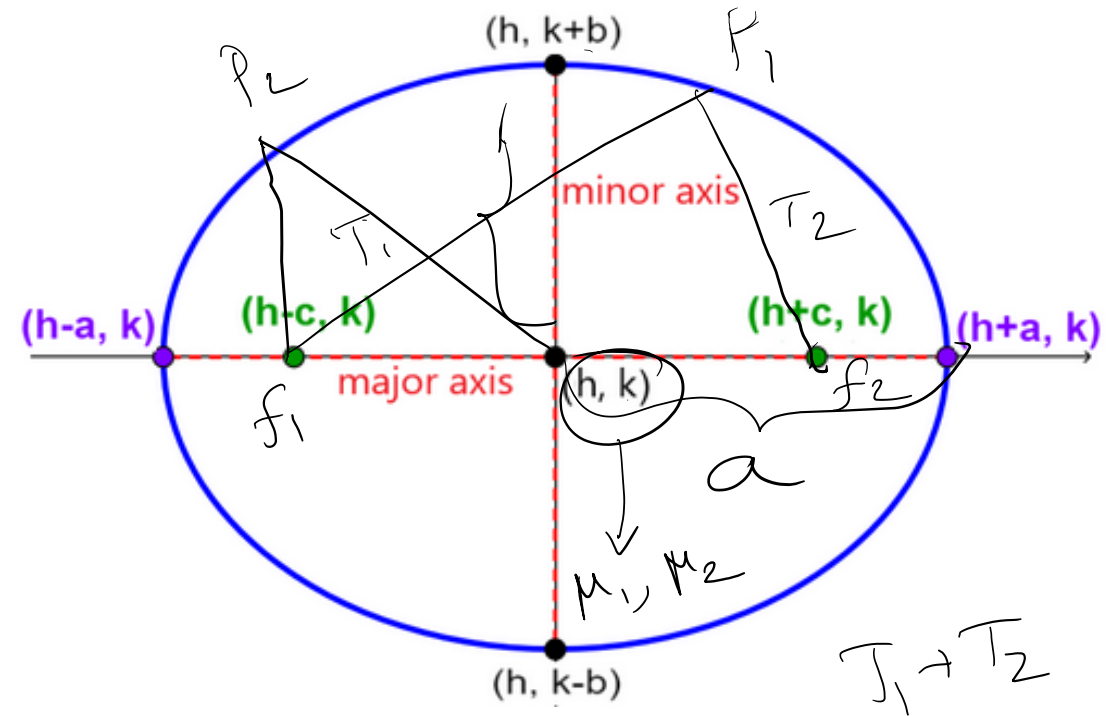


General Equation of an Ellipse

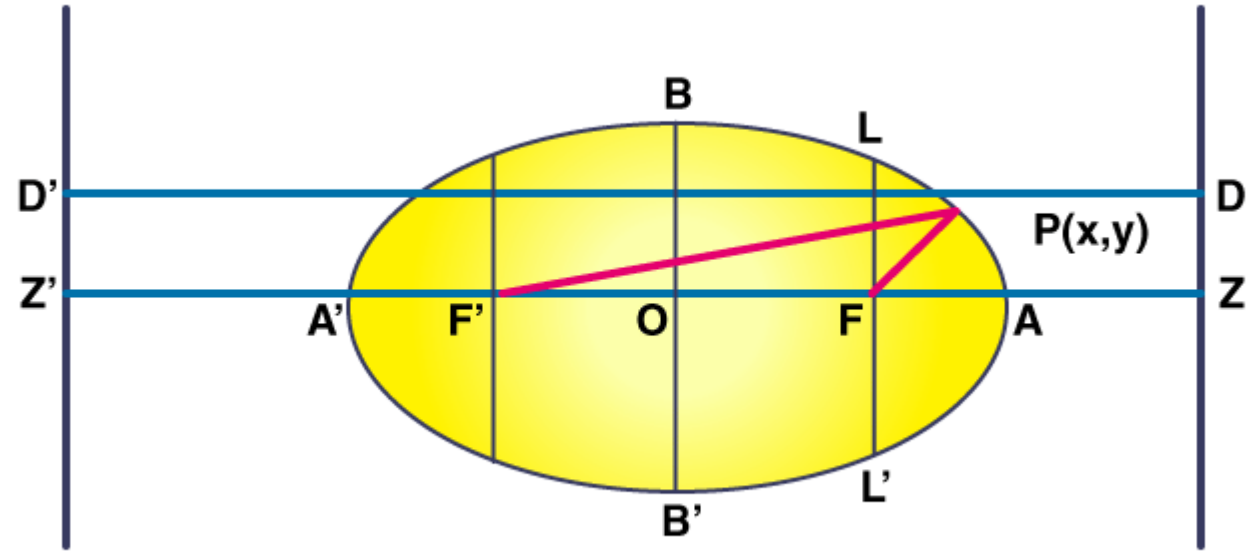
$$\frac{(x-h)^2}{a^2} + \frac{(y-k)^2}{b^2} = 1$$

where,

- $a > b$
- The major axis measures $2a$
- The minor axis measures $2b$
- The vertices are located at $(h \pm a, k)$
- The covertices are located at $(h, k \pm b)$
- The foci are located at $(h \pm c, k)$, where, $c^2 = a^2 - b^2$



Significance of Foci



The foci (plural of focus) of an ellipse are two special reference points located along its major axis. Their fundamental mathematical significance is that the **sum of the distances** from these two points to any single point on the edge of the ellipse is always a constant

Some Essential Background

❖ What will happen to the above equations if the center of the ellipse is **NOT** at the origin (0, 0)?

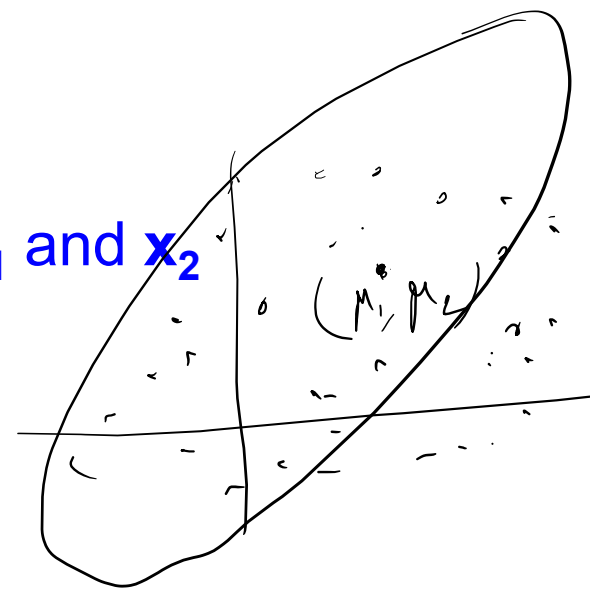
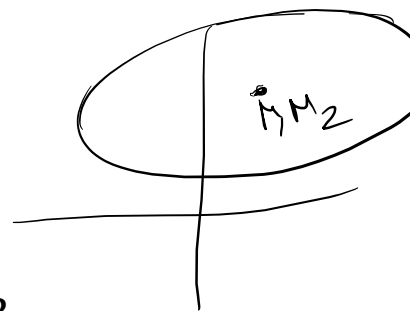
❖ Let the center be at $\mu = [\mu_1 \ \mu_2]$: We center around the mean of x_1 and x_2

❖ Then, the transformed variables are

No Covariance?

$$\frac{(x_1 - \mu_1)^2}{\sigma_{11}} + \frac{(x_2 - \mu_2)^2}{\sigma_{22}} = d^2$$

(x₁, x₂)
(μ₁, μ₂)

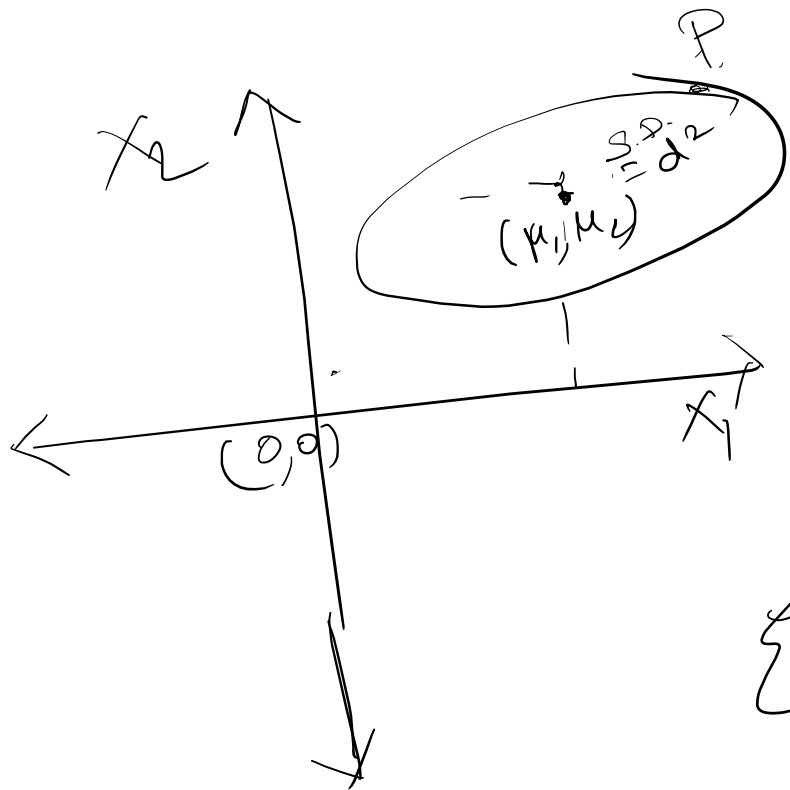


$$\Sigma = \begin{bmatrix} \sigma_{11} & 0 \\ 0 & \sigma_{22} \end{bmatrix}$$

$$\Sigma = \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix}$$

❖ In matrix notation we have

$$(\mathbf{x} - \mu)' \Sigma^{-1} (\mathbf{x} - \mu) = d^2$$



$$\frac{(x_1 - \mu_1)^2}{\sigma_{11}} + \frac{(x_2 - \mu_2)^2}{\sigma_{22}} = d^2$$

(d_s) Statistical distance

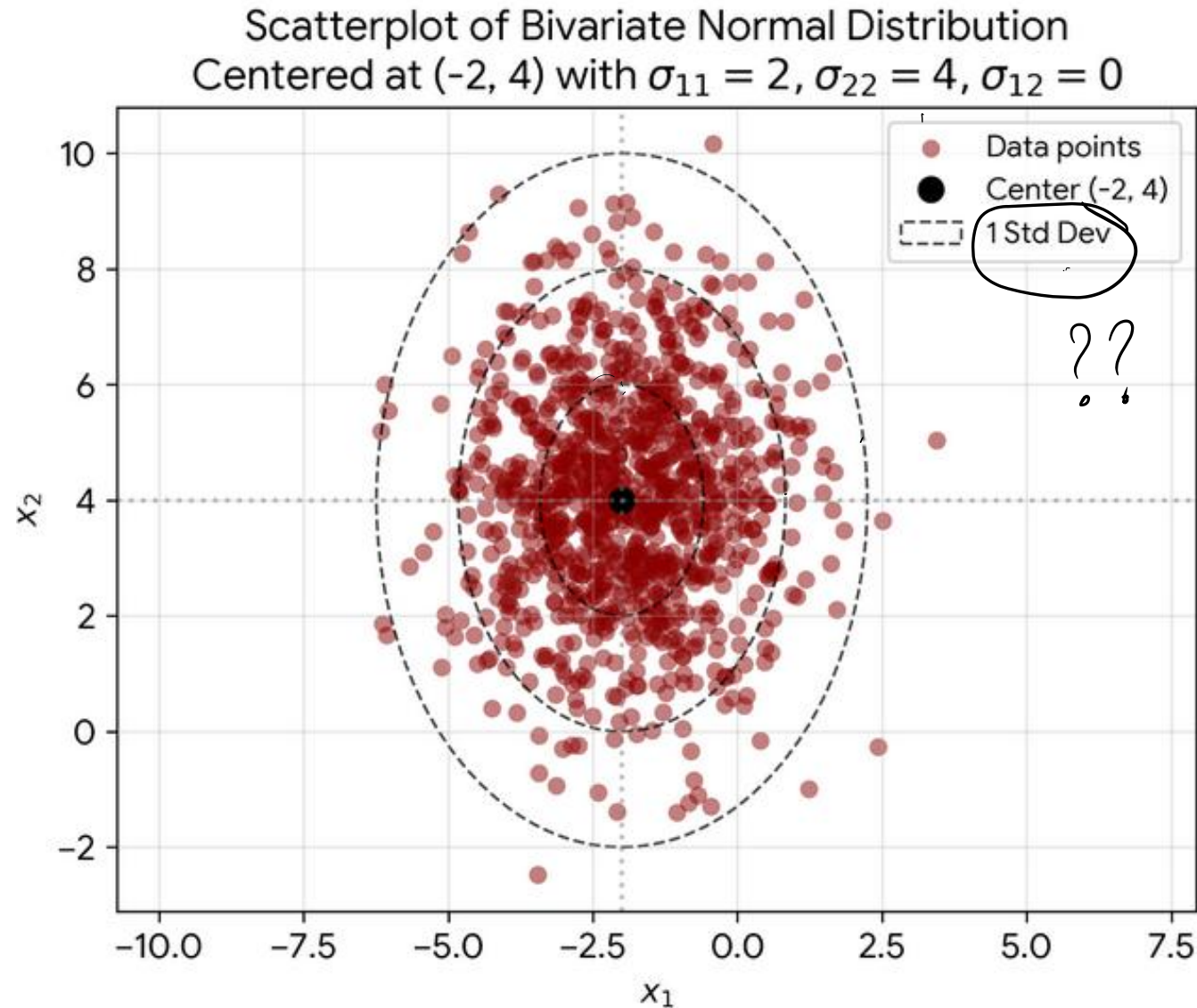
Euclidean Distance d_E^2

Euclidean

$$(x_1 - \mu_1)^2 + (x_2 - \mu_2)^2 = d_E^2$$

obviously $dE^2 \neq dS^2$ here

Centering Scatter Plot at any (μ_1, μ_2) ; No Correlation



"Other means"??

agents?

$$\frac{x_1^2}{\sigma_{11}} + \frac{x_2^2}{\sigma_{22}} = d_1^2$$

Handwritten calculations below the equation:

σ_{11} has values 2, 8, 4 written below it.

σ_{22} has values 4, 16, 8 written below it, with the 8 crossed out.

An ellipse centered at $(-2, 4)$ with a variance of 2 for X_1 and 4 for X_2 can be described mathematically by its standard bivariate normal contour equation (assuming zero covariance between X_1 and X_2):

$$\frac{(X_1 - h)^2}{\sigma_1^2 c^2} + \frac{(X_2 - k)^2}{\sigma_2^2 c^2} = 1$$

Substituting the given values:

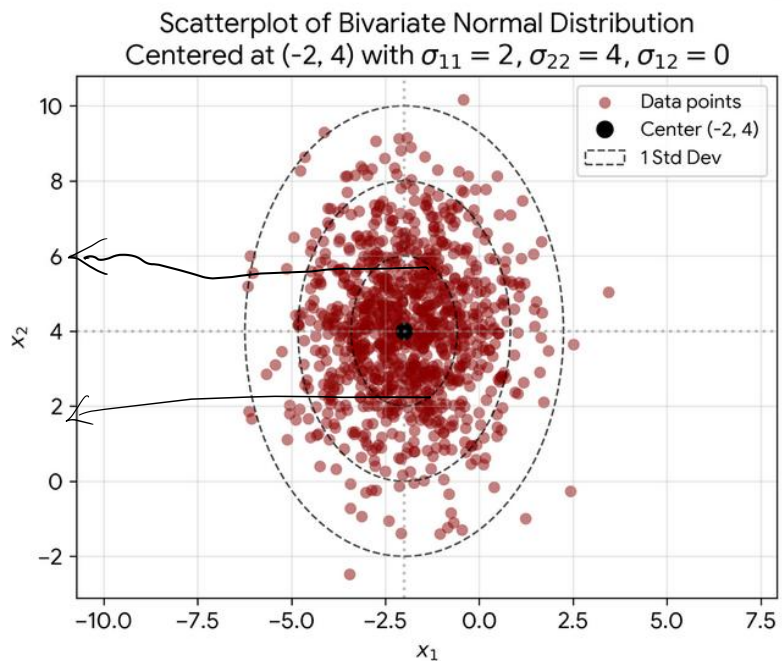
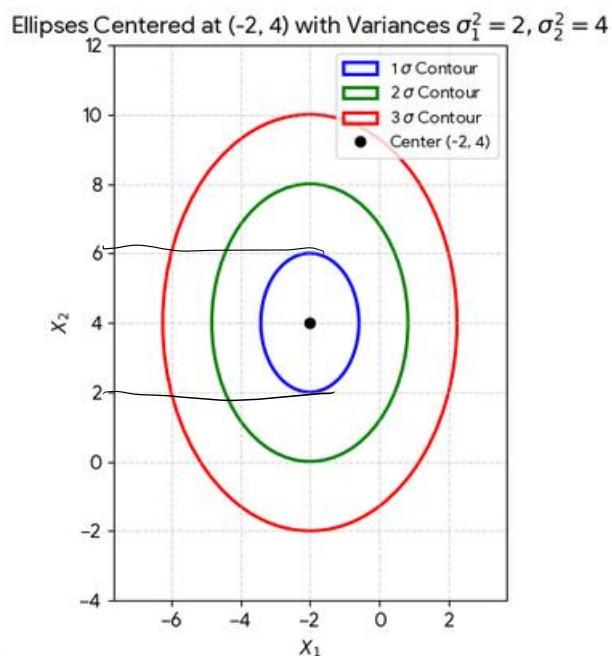
- Center $(h, k) = (-2, 4)$
- Variance along X_1 ($\sigma_1^2 = 2$) $\implies$ Semi-axis length along $X_1 = \sqrt{2} \approx 1.414$
- Variance along X_2 ($\sigma_2^2 = 4$) $\implies$ Semi-axis length along $X_2 = \sqrt{4} = 2$

The equation for the 1-standard deviation (1σ) contour ($c = 1$) is:

$$\frac{(X_1 + 2)^2}{2} + \frac{(X_2 - 4)^2}{4} = 1$$

The visualization above plots the contours for 1σ , 2σ , and 3σ (where $c = 1, 2, 3$ respectively):

- 1 σ Contour (Blue):** Spans from $-2 - \sqrt{2}$ to $-2 + \sqrt{2}$ horizontally, and from 2 to 6 vertically.
- 2 σ Contour (Green):** Spans from $-2 - 2\sqrt{2}$ to $-2 + 2\sqrt{2}$ horizontally, and from 0 to 8 vertically.
- 3 σ Contour (Red):** Spans from $-2 - 3\sqrt{2}$ to $-2 + 3\sqrt{2}$ horizontally, and from -2 to 10 vertically.



Matlab Code: Centering of Scatter Plot at any (μ_1, μ_2)

```
% Set random seed for reproducibility
rng(42);

% Parameters
mu = [-2, 4];           % Center / Mean
sigma11 = 2;            % Variance of X1
sigma22 = 4;            % Variance of X2
covariance = 0;         % Covariance between X1 and X2

% Construct the covariance matrix
Sigma = [sigma11, covariance; covariance, sigma22];

% Generate 1000 data points
num_points = 1000;
data = mvnrnd(mu, Sigma, num_points);
x1 = data(:, 1);
x2 = data(:, 2);

% Create the plot
figure;
hold on;

% 1. Plot the simulated data points
scatter(x1, x2, 25, [0.6, 0, 0], 'filled', 'MarkerFaceAlpha', 0.5);

% 2. Plot the center point
plot(mu(1), mu(2), 'ko', 'MarkerSize', 10, 'MarkerFaceColor', 'k');

% 3. Draw standard deviation contours (Ellipses)
theta = linspace(0, 2*pi, 100);
for n_std = 1:3
    % Scale the standard deviations by the number of deviations
    r1 = n_std * sqrt(sigma11);
    r2 = n_std * sqrt(sigma22);

    % Parametric equations for the ellipse centered at (-2, 4)
    ellipse_x = mu(1) + r1 * cos(theta);
    ellipse_y = mu(2) + r2 * sin(theta);

    if n_std == 1
        plot(ellipse_x, ellipse_y, 'k--', 'LineWidth', 1.5, 'DisplayName', '1, 2, 3 Std Dev Contours');
    else
        plot(ellipse_x, ellipse_y, 'k--', 'LineWidth', 1.5, 'HandleVisibility', 'off');
    end
end

% Formatting the plot
grid on;
ax = gca;
ax.GridAlpha = 0.3;
axis equal; % Ensures the 1:1 aspect ratio so the geometric shapes look correct

% Add reference lines for the center
xline(mu(1), ':', 'Color', [0.5, 0.5, 0.5], 'LineWidth', 1, 'HandleVisibility', 'off');
yline(mu(2), ':', 'Color', [0.5, 0.5, 0.5], 'LineWidth', 1, 'HandleVisibility', 'off');

% Labels and Title
xlabel('x_1');
ylabel('x_2');
title({'Scatterplot of Bivariate Normal Distribution', ...
    'Centered at (-2, 4) with \sigma_{11}=2, \sigma_{22}=4, \sigma_{12}=0'});
legend('Data points', 'Center (-2, 4)', 'Location', 'best');

hold off;
```

Python Code: Centering of Scatter Plot at any (μ_1, μ_2)

```
import numpy as np
from matplotlib import pyplot as plt
from matplotlib.patches import Ellipse
# Set random seed for statistical reproducibility

np.random.seed(42)

# Define distribution parameters

mean = [-2, 4]          # Center (mu_1, mu_2)

sigma_11 = 2             # Variance along x_1 axis
sigma_22 = 4             # Variance along x_2 axis
covariance = 0           # Covariance (sigma_12)

# Define the full covariance matrix
cov_matrix = [[sigma_11, covariance],
               [covariance, sigma_22]]

# Generate 1,000 random data points from the distribution
x1, x2 = np.random.multivariate_normal(mean, cov_matrix, 1000).T

# 1. Plot the random data points
plt.scatter(x1, x2, alpha=0.5, color='darkred', edgecolors='none', label='Data
points')

# 2. Plot the center point
plt.plot(mean[0], mean[1], 'ko', markersize=8, label='Center (-2, 4)')

# 3. Overlay the theoretical standard deviation contours (Ellipses)
ax = plt.gca()

for n_std in [1, 2, 3]:
    # Width and height correspond to full diameter (2 * radius)
    # Since covariance is zero, radii scale directly with sqrt(variance)
```

```
width = 2 * n_std * np.sqrt(sigma_11)
height = 2 * n_std * np.sqrt(sigma_22)

    ellipse = Ellipse(xy=mean, width=width, height=height,
                      edgecolor='black', linestyle='--', fill=False, alpha=0.7,
                      label=f'{n_std} Std Dev Contour' if n_std == 1 else "")

    ax.add_patch(ellipse)

# Format axes and labels
plt.xlabel('$x_1$')
plt.ylabel('$x_2$')

plt.title('Scatterplot of Bivariate Normal Distribution\nCentered at (-2, 4) with
\\sigma_{11}=2, \\sigma_{22}=4, \\sigma_{12}=0$')

# Add crosshairs for the center point
plt.axhline(4, color='gray', linestyle=':', alpha=0.5)
plt.axvline(-2, color='gray', linestyle=':', alpha=0.5)

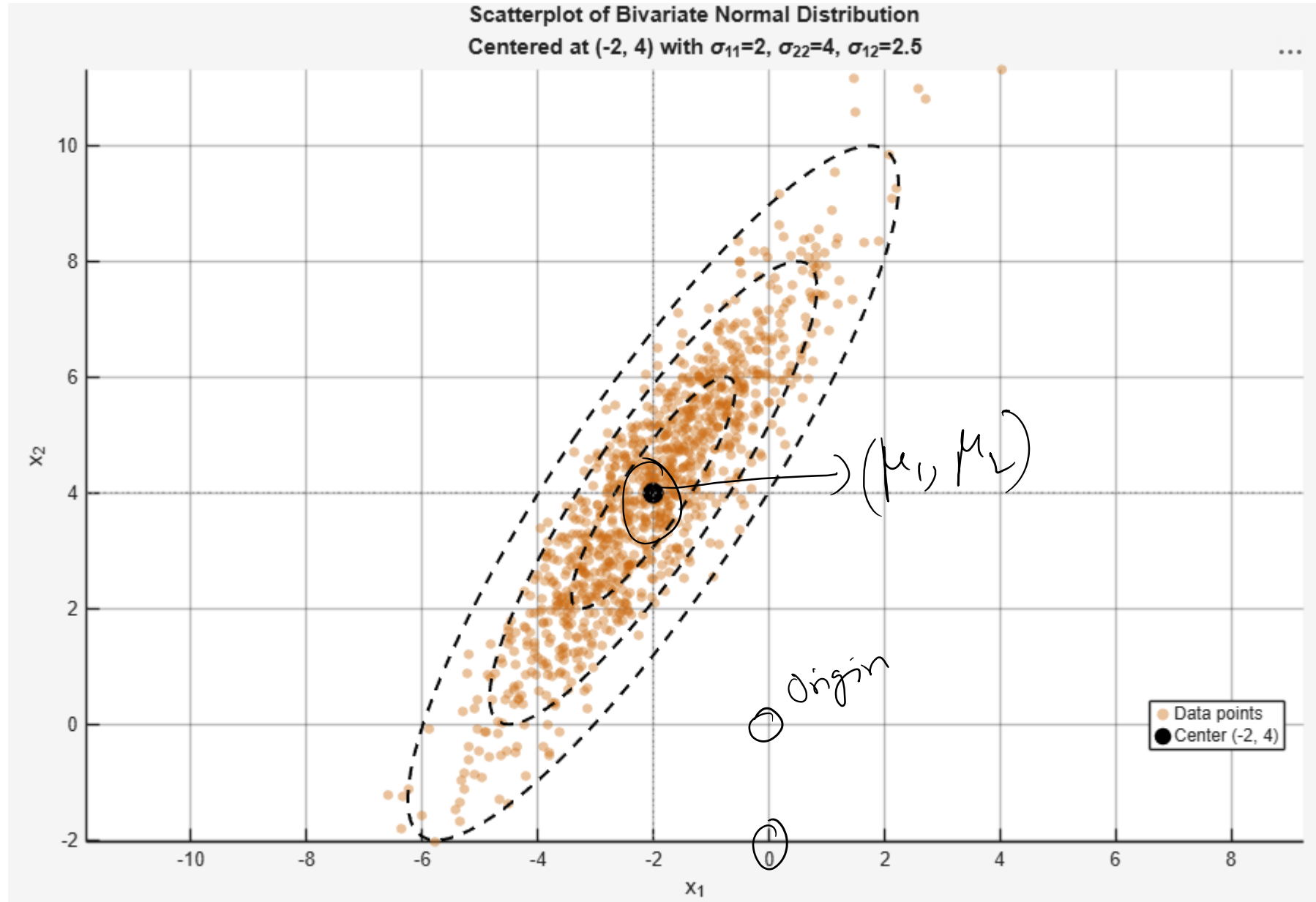
plt.grid(True, alpha=0.3)
plt.legend(loc='best')

# Crucial: Ensure 1:1 aspect ratio so the ellipse is not visually distorted
plt.axis('equal')

# Save the plot
plt.savefig('scatterplot_ellipse.png', bbox_inches='tight', dpi=300)

# plt.show() # Uncomment this line if running in a local GUI environment
```

General Ellipse with non-zero centre and with Correlation



$$\Sigma = \begin{bmatrix} 2 & 2.5 \\ 2.5 & 4 \end{bmatrix}$$

$$\sigma_{11} = 2$$

$$\sigma_{12} = 2.5$$

$$\sigma_{21} = 2.5$$

$$\sigma_{22} = 4$$

Matlab Code

```
% Set random seed for reproducibility
```

```
rng(42);
```

```
% Parameters
```

```
mu = [-2, 4];           % Center / Mean
```

```
sigma11 = 2;            % Variance of X1
```

```
sigma22 = 4;            % Variance of X2
```

```
covariance = 2.5;       % Covariance between X1 and X2
```

```
% Construct the covariance matrix
```

```
Sigma = [sigma11, covariance; covariance, sigma22];
```

```
% Generate 1000 data points from the bivariate normal distribution
```

```
num_points = 1000;
```

```
data = mvnrnd(mu, Sigma, num_points);
```

```
x1 = data(:, 1);
```

```
x2 = data(:, 2);
```

```
% Calculate Eigenvalues and Eigenvectors for the Ellipse Rotation
```

```
% The eigenvectors (V) determine the diagonal tilt angle.
```

```
% The square root of the eigenvalues (D) determine the lengths of the axes.
```

```
[V, D] = eig(Sigma);
```

```
% Create the plot
```

```
figure;
```

```
hold on;
```

```
% 1. Plot the simulated data points
```

```
scatter(x1, x2, 25, [0.8, 0.4, 0], 'filled', 'MarkerFaceAlpha', 0.4);
```

```
% 2. Plot the center point
```

```
plot(mu(1), mu(2), 'ko', 'MarkerSize', 10, 'MarkerFaceColor', 'k');
```

```
% 3. Draw tilted standard deviation contours (Ellipses)
```

```
theta = linspace(0, 2*pi, 100);
```

```
base_ellipse = [cos(theta); sin(theta)]; % Unit circle template
```

```
for n_std = 1:3
```

```
    % Scale the unit circle along its principal components using eigenvalues
```

```
    scaled_ellipse = V * (n_std * sqrt(D) * base_ellipse);
```

```
    % Shift the rotated ellipse to the center (-2, 4)
```

```
    ellipse_x = mu(1) + scaled_ellipse(1, :);
```

```
    ellipse_y = mu(2) + scaled_ellipse(2, :);
```

```
    if n_std == 1
```

```
        plot(ellipse_x, ellipse_y, 'k--', 'LineWidth', 1.5, 'DisplayName', '1, 2, 3 Std Dev Contours');
```

```
    else
```

```
        plot(ellipse_x, ellipse_y, 'k--', 'LineWidth', 1.5, 'HandleVisibility', 'off');
```

```
    end
```

```
end
```

```
% Formatting the plot
```

```
grid on;
```

```
ax = gca; ax.GridAlpha = 0.3;
```

```
axis equal; % Essential to see the true physical angle and geometry
```

```
% Add reference lines for the center
```

```
xline(mu(1), ':', 'Color', [0.5, 0.5, 0.5], 'LineWidth', 1, 'HandleVisibility', 'off');
```

```
yline(mu(2), ':', 'Color', [0.5, 0.5, 0.5], 'LineWidth', 1, 'HandleVisibility', 'off');
```

```
% Labels and Title
```

```
xlabel('x_1'); ylabel('x_2');
```

```
title(['Scatterplot of Bivariate Normal Distribution', ...
```

```
    'Centered at (-2, 4) with \sigma_{11}=2, \sigma_{22}=4, \sigma_{12}=2.5']);
```

```
legend('Data points', 'Center (-2, 4)', 'Location', 'best');
```

```
hold off;
```

Python Code

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Ellipse
```

```
# Set random seed for reproducibility
np.random.seed(42)
```

```
# Define distribution parameters
mean = [-2, 4]          # Center (mu_1, mu_2)
sigma_11 = 2            # Variance along x_1 axis
sigma_22 = 4            # Variance along x_2 axis
covariance = 2.5        # Covariance (sigma_12)
```

```
# Define the full covariance matrix
cov_matrix = [[sigma_11, covariance],
              [covariance, sigma_22]]
```

```
# Generate 1,000 random data points from the distribution
x1, x2 = np.random.multivariate_normal(mean, cov_matrix, 1000).T
```

```
# Initialize the plot
plt.figure(figsize=(6, 6))
```

```
# 1. Plot the random data points
plt.scatter(x1, x2, alpha=0.5, color='darkorange', edgecolors='none', label='Data points')
```

```
# 2. Plot the center point
plt.plot(mean[0], mean[1], 'ko', markersize=8, label='Center (-2, 4)')
```

```
# 3. Calculate and overlay the tilted standard deviation contours (Ellipses)
# Get eigenvalues and eigenvectors to find the tilt angle and scaling factors
vals, vecs = np.linalg.eigh(cov_matrix)
```

```
# Sort eigenvalues and eigenvectors in descending order to align with the major axis
order = vals.argsort()[::-1]
vals, vecs = vals[order], vecs[:, order]
```

```
# Compute the rotation angle in degrees
angle = np.degrees(np.arctan2(vecs[1, 0], vecs[0, 0]))
```

```
ax = plt.gca()
for n_std in [1, 2, 3]:
    # Width and height correspond to full diameters along the principal axes
    # (2 * number of standard deviations * sqrt(eigenvalue))
    width = 2 * n_std * np.sqrt(vals[0])
    height = 2 * n_std * np.sqrt(vals[1])
```

```
    ellipse = Ellipse(xy=mean, width=width, height=height, angle=angle,
                      edgecolor='black', linestyle='--', fill=False, alpha=0.7,
                      label=f'{n_std} Std Dev Contour' if n_std == 1 else "")
    ax.add_patch(ellipse)
```

```
# Format axes and labels
plt.xlabel('$x_1$')
plt.ylabel('$x_2$')
plt.title('Scatterplot of Bivariate Normal Distribution\nCentered at (-2, 4) with\n\\sigma_{11}=2, \\sigma_{22}=4, \\sigma_{12}=2.5$')
```

```
# Add crosshairs for the center point
plt.axhline(4, color='gray', linestyle=':', alpha=0.5)
plt.axvline(-2, color='gray', linestyle=':', alpha=0.5)
```

```
plt.grid(True, alpha=0.3)
plt.legend(loc='best')
```

```
# Crucial: Ensure 1:1 aspect ratio so the diagonal tilt and shape are visually accurate
plt.axis('equal')
```

```
# Save and display the plot
plt.savefig('scatterplot_covariance_2.5.png', bbox_inches='tight', dpi=300)
# plt.show() # Uncomment this line if running in a local GUI environment
```


$$(\mathbf{x} - \boldsymbol{\mu})' \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) = d^2$$

$$\boldsymbol{\mu} = \begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}$$

- ❖ Here d^2 , is known as **squared statistical distance**
- ❖ Statistical distance d^2 **incorporates the variance-covariance** relationship of a given process and hence is very different from the Euclidean distance.
- ❖ It may be extended to any p number of variables ($p > 1$) *features*
- ❖ If there are n data points, we can calculate d_i^2 , $i=1,2,\dots,n$ using Equation below

Specific coordinate

$$\underline{(\mathbf{x}_i - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x}_i - \boldsymbol{\mu}) = d_i^2}$$

$$\begin{bmatrix} x_{i,1} - \mu_1 & x_{i,2} - \mu_2 \end{bmatrix} \underbrace{\begin{bmatrix} \sigma_{11} & \sigma_{12} \\ \sigma_{12} & \sigma_{22} \end{bmatrix}^{-1}} \begin{bmatrix} x_{i,1} - \mu_1 \\ x_{i,2} - \mu_2 \end{bmatrix} = d_i^2$$

Handwritten notes illustrating the calculation of d_i^2 for multiple data points:

- For $i=1$: (x_{11}, x_{12}) leads to d_1^2
- For $i=2$: (x_{21}, x_{22}) leads to d_2^2
- For $i=3$: (x_{31}, x_{32}) leads to d_3^2
- For $i=k$: (x_{k1}, x_{k2}) leads to d_k^2
- General form: $\mathbf{x}_i = \begin{bmatrix} x_{i1} \\ x_{i2} \end{bmatrix}$

Mahalanobis Distance

❖ But the population mean vector μ and covariance matrix Σ are seldom known.

❖ Then, **i/o**

we get

$$(\mathbf{x}_i - \mu)^T \Sigma^{-1} (\mathbf{x}_i - \mu) = d_i^2$$

Statistical distance

$$(\mathbf{x}_i - \bar{\mathbf{x}})' \mathbf{S}^{-1} (\mathbf{x}_i - \bar{\mathbf{x}}) = d_i^2$$

Sample

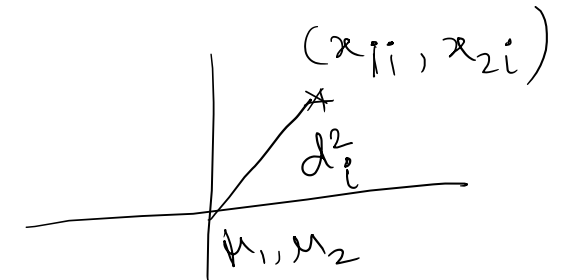
❖ Hence, we use their sample estimates $\bar{\mathbf{x}}$ and $\mathbf{S}$.

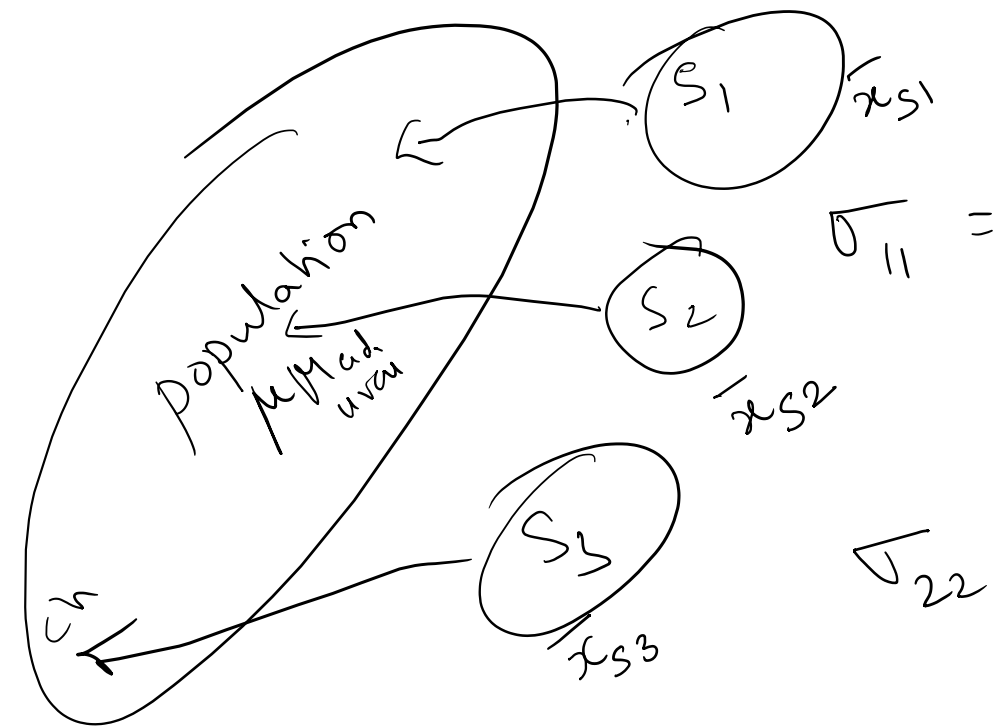
❖ The distance d_i^2 is known as squared **Mahalanobis distance** (MD) which was developed by Prasanta Chandra Mahalanobis in the 1930s.

National Statistics Day



Mahalanobis has been considered the father of statistics in India





$$\sigma_{11} =$$

$$\frac{\sum_{i=1}^n (x_{1i} - \mu_1)^2}{n-1}$$

$$\frac{\sum_{j=1}^n (x_{2j} - \mu_2)^2}{n-1}$$

$$\sigma_{22} =$$

population
 parameters
 μ_1, σ_{11}
 μ_2, σ_{22}

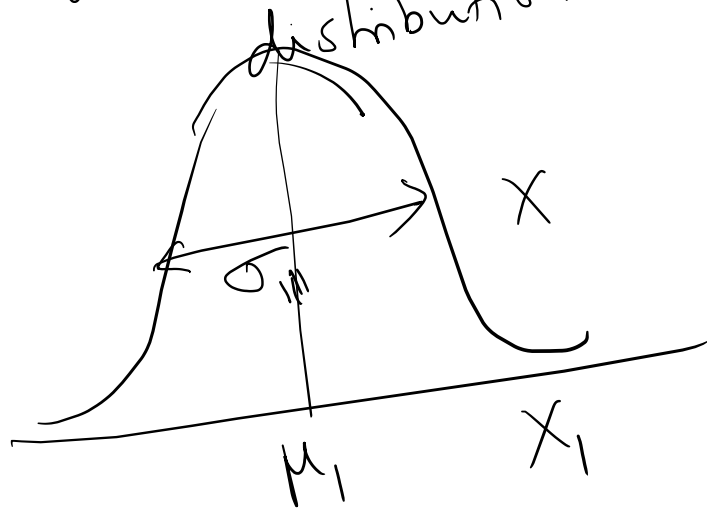
$$s_{11} = \frac{\sum_{i=1}^{n'} (x_{1i} - \bar{x}_1)^2}{n'-1}$$

$$s_{22} = \frac{\sum_{j=1}^{n'} (x_{2j} - \bar{x}_2)^2}{n'-1}$$

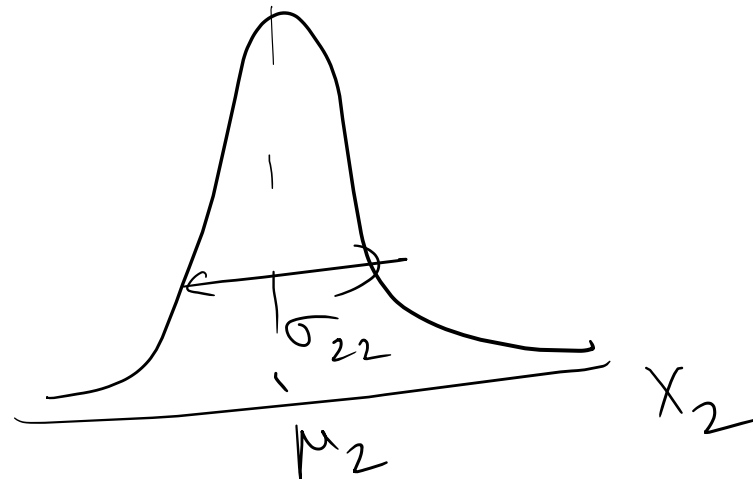
Sampling $\rightarrow$ Sample
 parameters.
 $\bar{x}_1, s_{11}$
 $\bar{x}_2, s_{22}$

- (A) $n < n'$
 (B) $n > n'$ ✓
 (C) $n = n'$
 (D) $n \neq n'$ uncorrelated

Univariate normal distribution



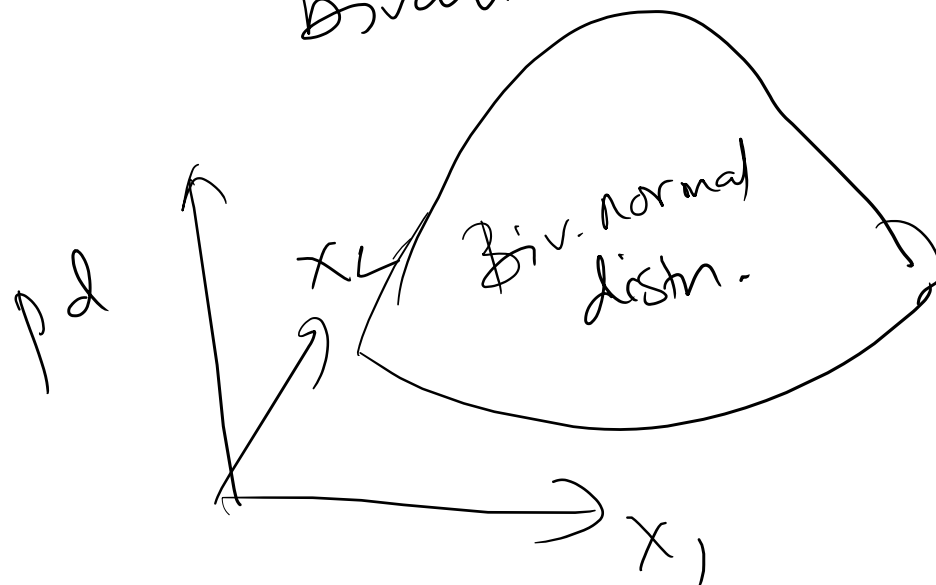
σ_{11}
 Measure of
 Variance
 (mov)



σ_{22} mov

X_1 bivariate
 X_2 normal
 distn.

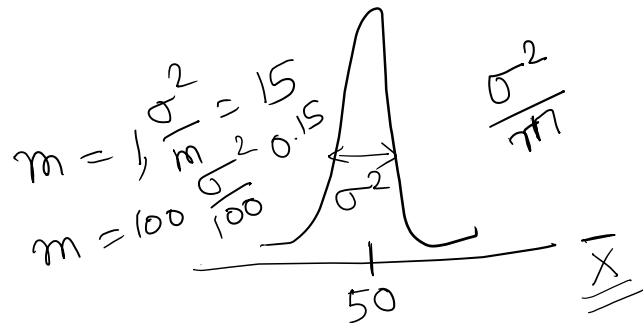
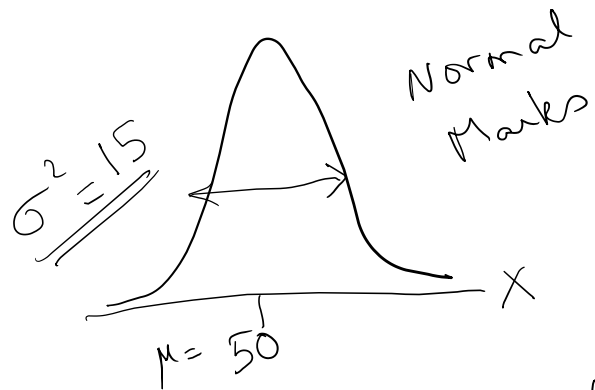
bivariate analysis.



X_1
 X_2
 $\vdots$
 X_p

Multivariate normal distn.

X^2 distn.
 Chi-Square



m - Sample Size

Univariate Distⁿ. Normal
 Bivariate Distⁿ.

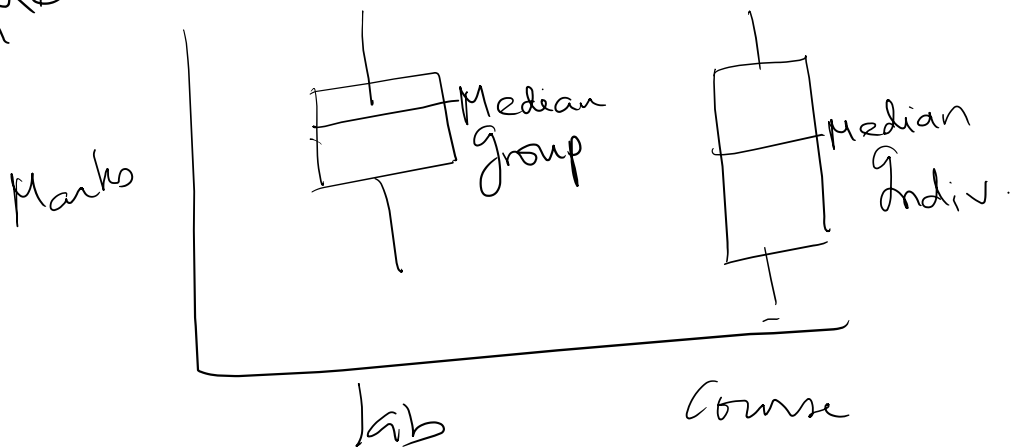
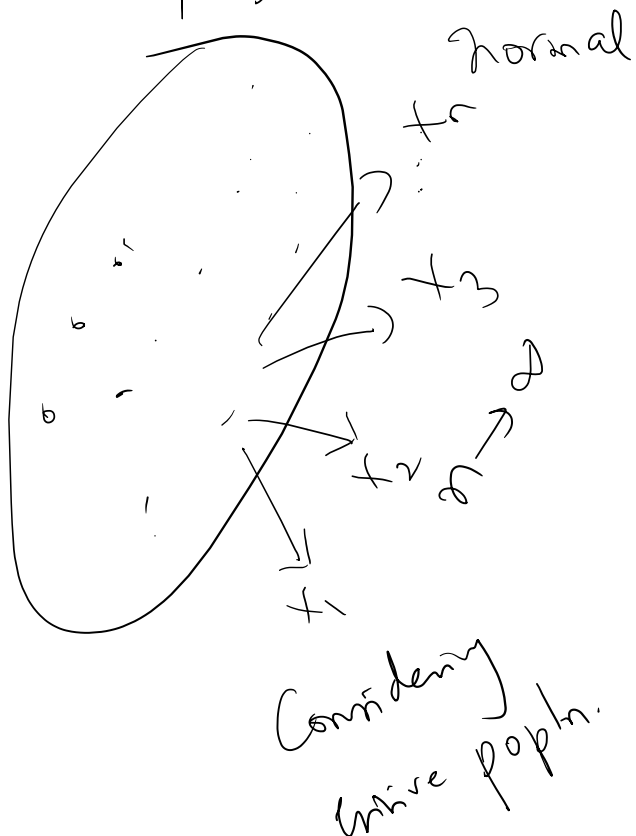
Sample mean $X_1 \sim N$
 $X_2 \sim N$

Sample I Mean $\frac{X_{I1} + X_{I2} + \dots + X_{I100}}{100}$ 55

Sample II Mean $\frac{X_{II1} + X_{II2} + \dots + X_{II100}}{100}$ 48

Sample III Mean $\frac{X_{III1} + X_{III2} + \dots + X_{III100}}{100}$

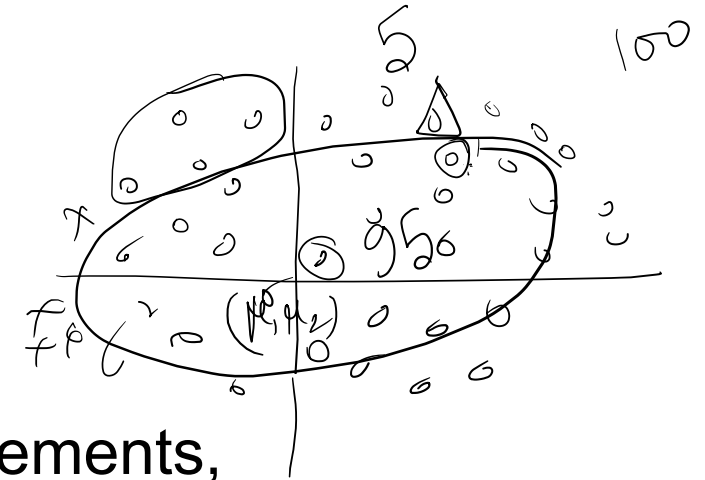
$\bar{X}_I$
 $\bar{X}_{II}$
 $\vdots$
 $\bar{X}_x$



Applications of Mahalanobis Distance

Mahalanobis distance plays a very important role in multivariate statistics and has several multivariate data applications such as

- (i) detection of multivariate outliers,
- (ii) quality control,
- (iii) choice of calibration samples for a large set of measurements,
- (iv) clustering based pattern recognition,
- (v) Detection of faults



Applications of Mahalanobis Distance

- ❖ The selection of calibration samples from a large set of measurements and
- ❖ for investigating the representativity between two data sets.
- ❖ In pattern recognition, the MD is applied in clustering techniques such as the k-Nearest Neighbour method,
- ❖ In discrimination techniques such as linear, quadratic and regularized discriminant analysis DA,

Need for Statistical Distance

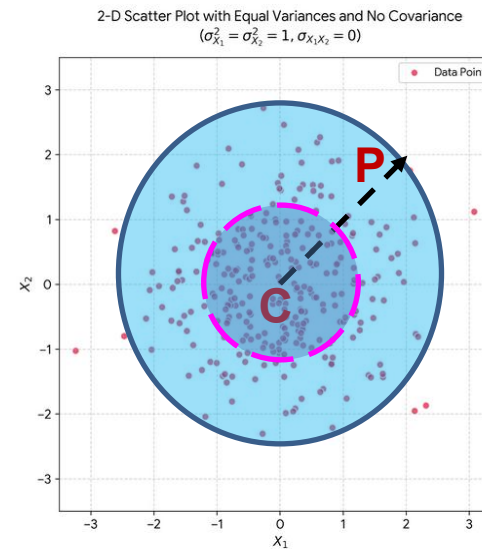
- ❖ Consider the observation $\mathbf{x} = [x_1, x_2]^T$, denoted as $\underline{P}(x_1, x_2)$.
- ❖ In all the three applications shown below I, II, and III, the Euclidean distance $d(OP)$ is same,
- ❖ We ask whether the observation $P(x_1, x_2)$ represents the same relevance for all the three applications?

Inlier

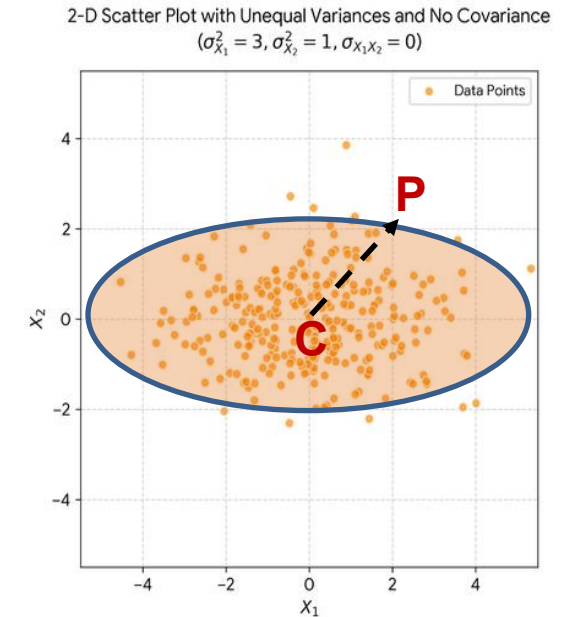
Outlier

Need for Statistical Distance

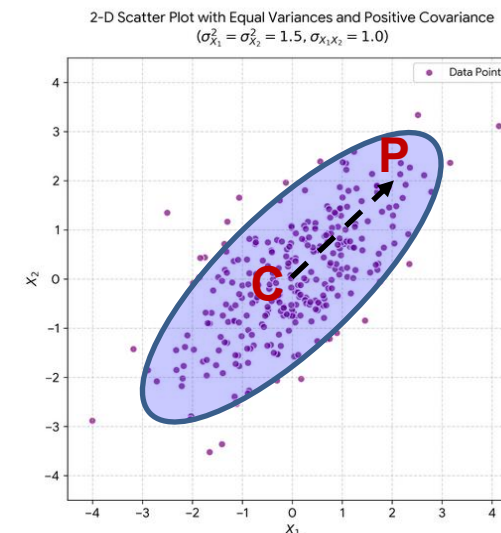
- ❖ For the first and second case on RHS, the observations $\mathbf{P}(x_1, x_2)$, both equi-distant from the origin are just on the boundary and you need to check if they are outliers or not
- ❖ Had the pink circle applied, $\mathbf{P}$ would be an outlier
- ❖ However, in the third case, the point $\mathbf{P}$ though equidistant from the origin is within the crowd and is NOT an outlier
- ❖ Hence, the location of the point within or out of the crowd depends not only on the Euclidean distance, but also on the shape of the curve (ellipse or circle) and the inclination all of which are decided by the variance-covariance matrix Σ or $\mathbf{S}$.



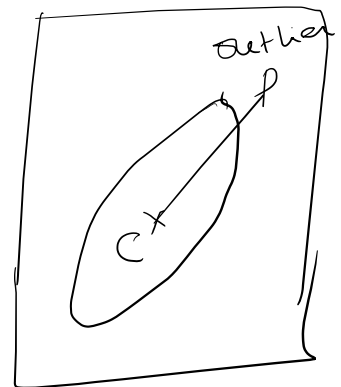
I



II



III



Application I Analysis

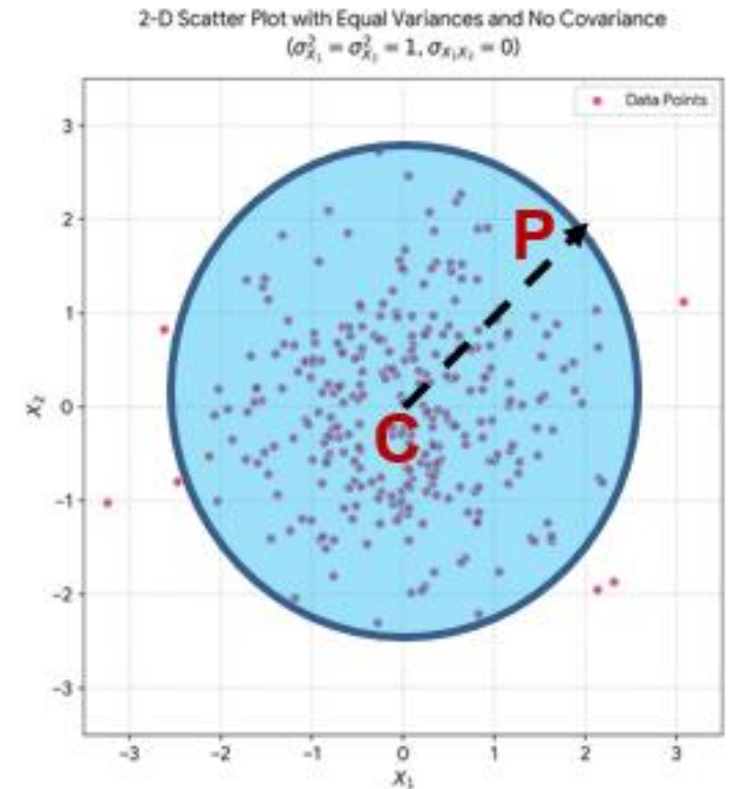
$$\frac{x_1^2}{\sigma_{11}} + \frac{x_2^2}{\sigma_{22}} = d^2$$

$$x_1^{*2} + x_2^{*2} = d^2$$

For the special case, where both variances σ_{11} and σ_{22} are unity and the correlation does not exist we have

$$\frac{x_1^2}{(\sqrt{1})^2} + \frac{x_2^2}{(\sqrt{1})^2} = d^2$$

Locus of points governed by the equation results in **CIRCLE** centred at the origin and statistical distance reduces to Euclidean Distance



$$\sigma_{11} = \sigma_{22} \quad \text{No Covariance}$$

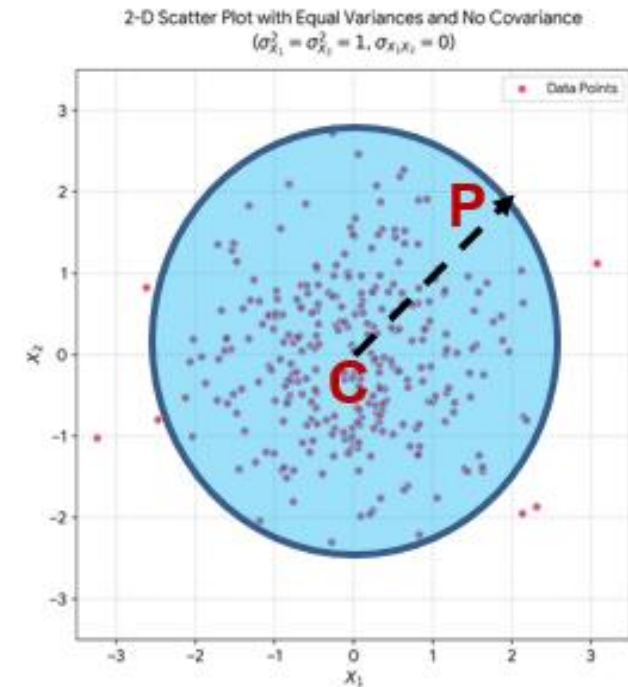
Application I Analysis

For the special case, where both variances σ_{11} and σ_{22} are unity and the correlation does not exist we have

$$\frac{x_1^2}{(\sqrt{1})^2} + \frac{x_2^2}{(\sqrt{1})^2} = d^2$$

Locus of points governed by the equation results in **CIRCLE** centred at the origin

When you calculate statistical distance with unity variances and zero correlation, the covariance matrix simplifies to the identity matrix. Because of this, the formula reduces directly to standard Euclidean distance.



The general formula for statistical distance is:

$$D = \sqrt{(x - \mu)^T \Sigma^{-1} (x - \mu)}$$

Where:

- x is your data point
- μ is the mean
- Σ is the covariance matrix

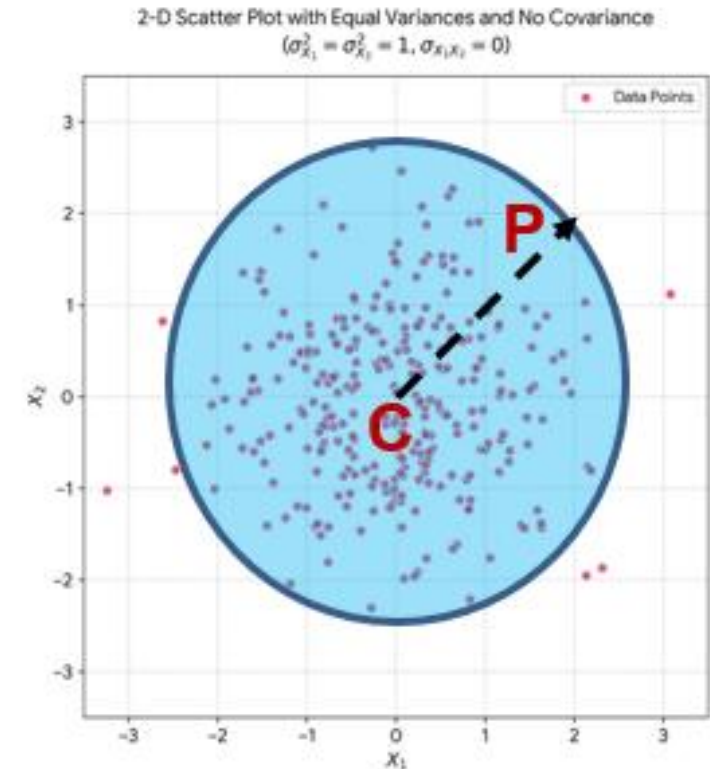
Because variances are 1 and correlations are 0, the covariance matrix Σ is simply the identity matrix, I . Therefore, its inverse, Σ^{-1} , is also the identity matrix, and the formula simplifies to:

$$D = \sqrt{(x - \mu)^T I (x - \mu)} = \sqrt{(x - \mu)^T (x - \mu)}$$

Application I Analysis

Geometrically, this means the set of points at a constant statistical distance from the mean forms a circle (in 2D space) or a hypersphere (in higher dimensions) where

the Euclidean distance is the radius



Application II Analysis: No Correlation, Different Variances

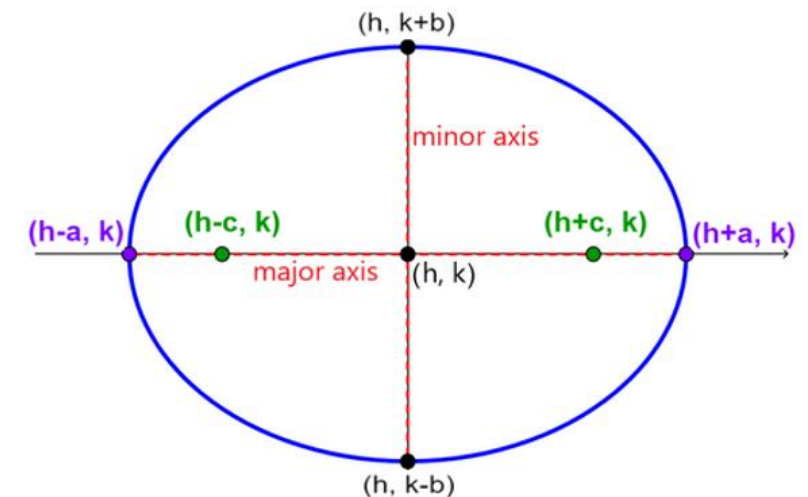
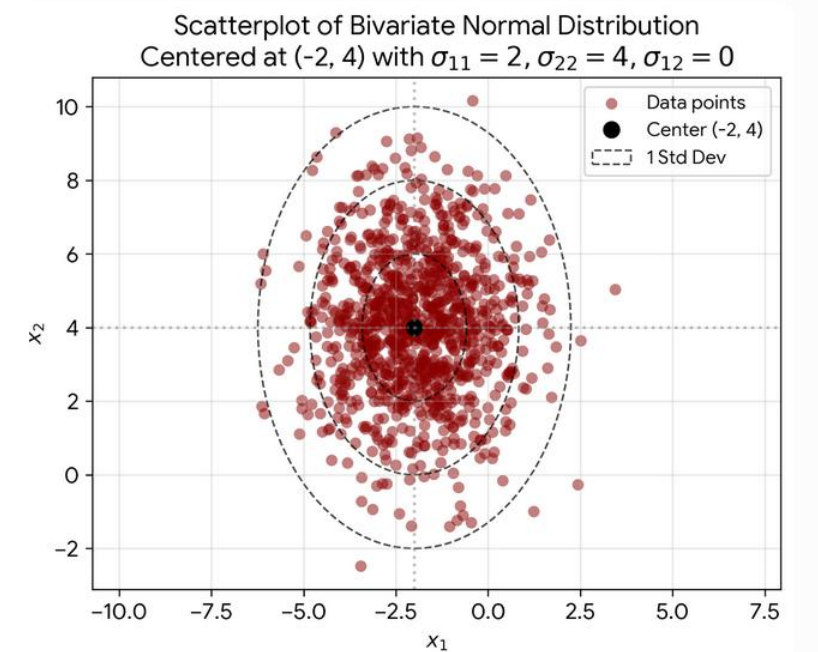
$$(\mathbf{x} - \boldsymbol{\mu})' \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) = d^2$$

$$\begin{bmatrix} x_{i,1} - \mu_1 & x_{i,2} - \mu_2 \end{bmatrix} \begin{bmatrix} 1/\sigma_{11} & 0 \\ 0 & 1/\sigma_{22} \end{bmatrix} \begin{bmatrix} x_{i,1} - \mu_1 \\ x_{i,2} - \mu_2 \end{bmatrix} = \underline{\underline{d_i^2}}$$

$$\frac{(x_{i,1} - \mu_1)^2}{(\sqrt{\sigma_{11}} d_i)^2} + \frac{(x_{i,2} - \mu_2)^2}{(\sqrt{\sigma_{22}} d_i)^2} = 1$$

Locus of points governed by the equation
results in **Ellipse** centred at (μ_1, μ_2)

$$\frac{(x - h)^2}{a^2} + \frac{(y - k)^2}{b^2} = 1$$



Application II Analysis: No Correlation, Different Variances

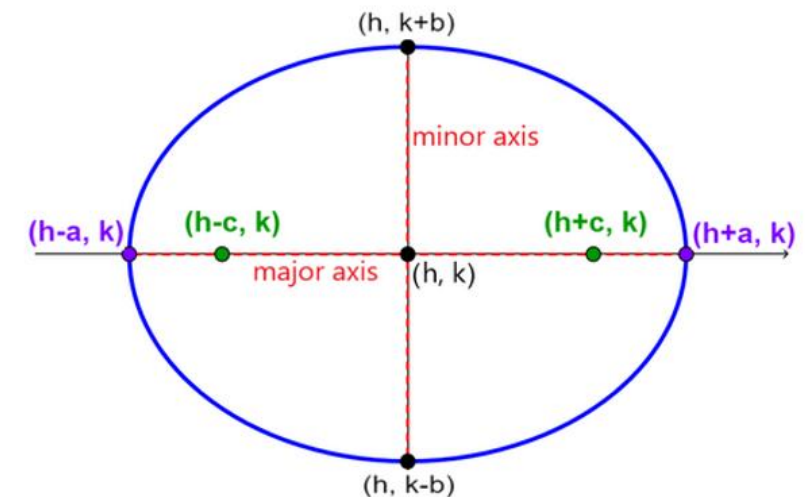
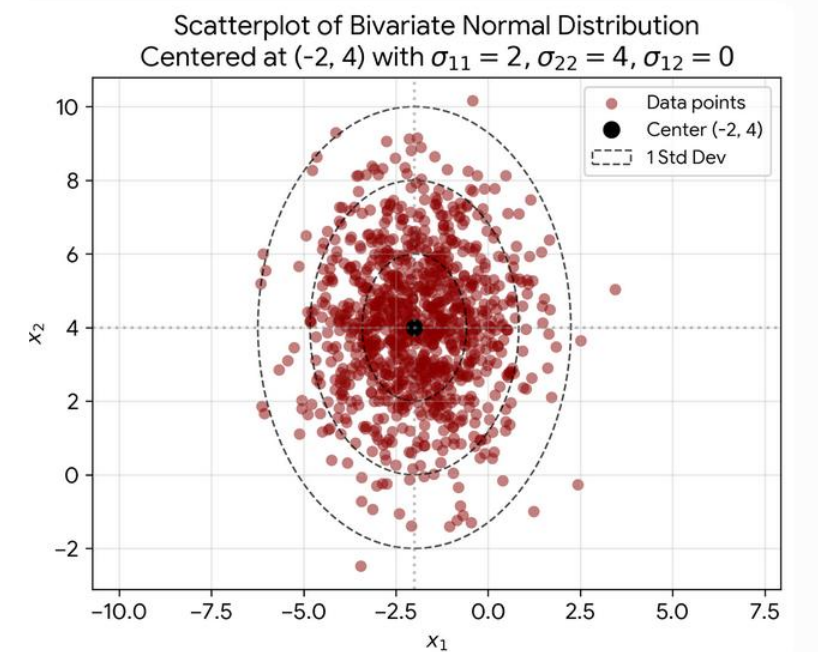
$$(\mathbf{x} - \boldsymbol{\mu})' \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) = d^2$$

$$\begin{bmatrix} x_{i,1} - \mu_1 & x_{i,2} - \mu_2 \end{bmatrix} \begin{bmatrix} 1/\sigma_{11} & 0 \\ 0 & 1/\sigma_{22} \end{bmatrix} \begin{bmatrix} x_{i,1} - \mu_1 \\ x_{i,2} - \mu_2 \end{bmatrix} = \underline{\underline{d_i^2}}$$

$$\frac{(x_{i,1} - \mu_1)^2}{(\sqrt{\sigma_{11}} d_i)^2} + \frac{(x_{i,2} - \mu_2)^2}{(\sqrt{\sigma_{22}} d_i)^2} = 1$$

Locus of points governed by the equation
results in **Ellipse** centred at (μ_1, μ_2)

$$\frac{(x - h)^2}{a^2} + \frac{(y - k)^2}{b^2} = 1$$

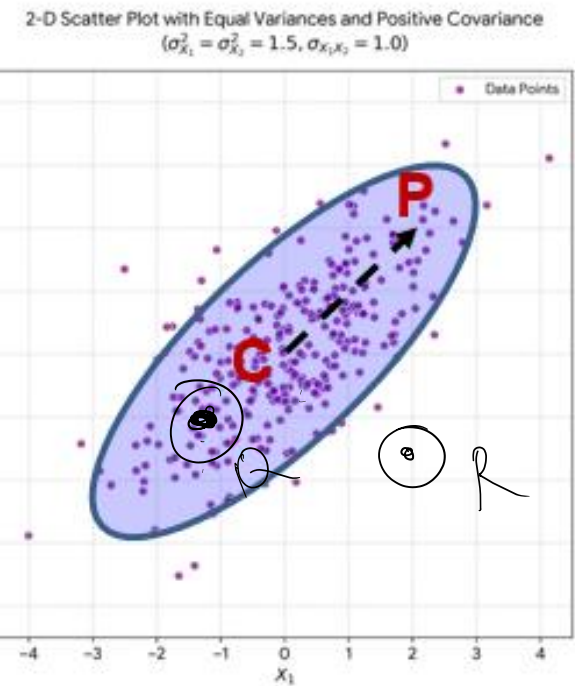


General Case: Centered at μ_1 and μ_2 with Correlation

$$\Sigma = \begin{bmatrix} \sigma_{11} & \sigma_{12} \\ \sigma_{12} & \sigma_{22} \end{bmatrix}$$

$$(\mathbf{x} - \boldsymbol{\mu})' \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) = d^2$$

$$\frac{(x_{i,1} - \mu_1)^2}{\sigma_{11}} + \frac{(x_{i,2} - \mu_2)^2}{\sigma_{22}} = d_i^2$$

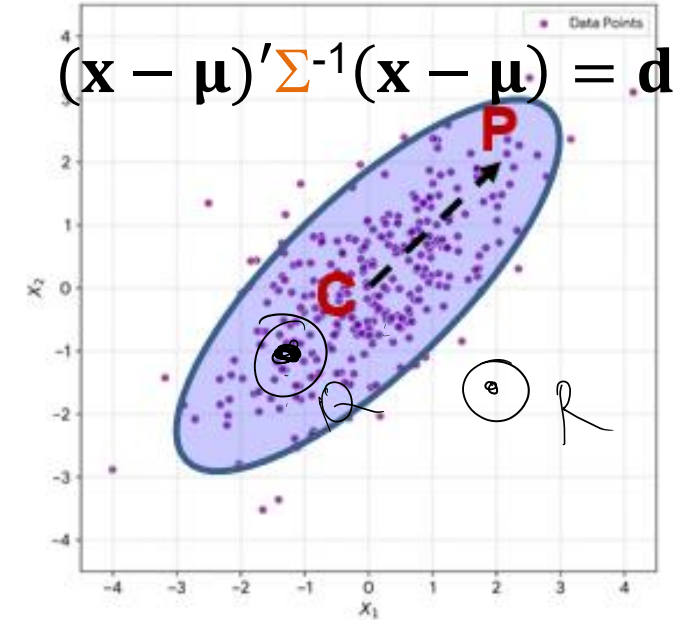


$$\begin{bmatrix} x_{i,1} - \mu_1 & x_{i,2} - \mu_2 \end{bmatrix} \begin{bmatrix} 1 \\ \sigma_{11}\sigma_{22} - \sigma_{12}^2 \end{bmatrix} \begin{bmatrix} \sigma_{22} & -\sigma_{12} \\ -\sigma_{12} & \sigma_{11} \end{bmatrix} \begin{bmatrix} x_{i,1} - \mu_1 \\ x_{i,2} - \mu_2 \end{bmatrix} = d_i^2$$

$$\begin{bmatrix} x_{i,1} - \mu_1 & x_{i,2} - \mu_2 \end{bmatrix} \begin{bmatrix} 1 \\ \sigma_{11}\sigma_{22} - \sigma_{12}^2 \end{bmatrix} \begin{bmatrix} \frac{\sigma_{22}}{\sigma_{11}\sigma_{22} - \sigma_{12}^2} & \frac{-\sigma_{12}}{\sigma_{11}\sigma_{22} - \sigma_{12}^2} \\ \frac{-\sigma_{12}}{\sigma_{11}\sigma_{22} - \sigma_{12}^2} & \frac{\sigma_{11}}{\sigma_{11}\sigma_{22} - \sigma_{12}^2} \end{bmatrix} \begin{bmatrix} x_{i,1} - \mu_1 \\ x_{i,2} - \mu_2 \end{bmatrix} = d_i^2$$

General Case: Centered at μ_1 and μ_2 with Correlation

2-D Scatter Plot with Equal Variances and Positive Covariance
($\sigma_{x_1}^2 = \sigma_{x_2}^2 = 1.5$, $\sigma_{x_1 x_2} = 1.0$)



$$(\mathbf{x} - \boldsymbol{\mu})' \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) = d^2$$

$$\begin{bmatrix} x_1 - \mu_1 & x_2 - \mu_2 \\ 1 \times 1 & 1 \times 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 - \mu_1 \\ x_2 - \mu_2 \end{bmatrix} = d^2$$

$$\begin{bmatrix} x_1 - \mu_1 & x_2 - \mu_2 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 - \mu_1 \\ x_2 - \mu_2 \end{bmatrix} = d^2$$

$$\begin{bmatrix} x_{i,1} - \mu_1 & x_{i,2} - \mu_2 \end{bmatrix} \begin{bmatrix} \frac{1}{\sigma_{11}\sigma_{22} - \sigma_{12}^2} \\ \frac{1}{\sigma_{11}\sigma_{22} - \sigma_{12}^2} \end{bmatrix} \begin{bmatrix} \frac{\sigma_{22}}{\sigma_{11}\sigma_{22} - \sigma_{12}^2} & \frac{-\sigma_{12}}{\sigma_{11}\sigma_{22} - \sigma_{12}^2} \\ \frac{-\sigma_{12}}{\sigma_{11}\sigma_{22} - \sigma_{12}^2} & \frac{\sigma_{11}}{\sigma_{11}\sigma_{22} - \sigma_{12}^2} \end{bmatrix} \begin{bmatrix} x_{i,1} - \mu_1 \\ x_{i,2} - \mu_2 \end{bmatrix} = d_i^2$$

General Case: A More Compact Version

$$\rho_{12} = \begin{bmatrix} 1 & \alpha \\ \alpha & 1 \end{bmatrix}$$

$$(\mathbf{x} - \boldsymbol{\mu})' \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) = d^2$$

$$[x_{i,1} - \mu_1 \quad x_{i,2} - \mu_2] \begin{bmatrix} 1 \\ \frac{-\sigma_{12}}{\sigma_{11}\sigma_{22} - \sigma_{12}^2} \end{bmatrix} \begin{bmatrix} \frac{\sigma_{22}}{\sigma_{11}\sigma_{22} - \sigma_{12}^2} & \frac{-\sigma_{12}}{\sigma_{11}\sigma_{22} - \sigma_{12}^2} \\ \frac{-\sigma_{12}}{\sigma_{11}\sigma_{22} - \sigma_{12}^2} & \frac{\sigma_{11}}{\sigma_{11}\sigma_{22} - \sigma_{12}^2} \end{bmatrix} \begin{bmatrix} x_{i,1} - \mu_1 \\ x_{i,2} - \mu_2 \end{bmatrix} = d_i^2$$

$$\begin{bmatrix} 1 \\ 1 - \rho_{12}^2 \end{bmatrix} \left[\frac{(x_{i,1} - \mu_1)^2}{(\sqrt{\sigma_{11}})^2} - 2\rho_{12} \frac{(x_{i,1} - \mu_1)(x_{i,2} - \mu_2)}{\sqrt{\sigma_{11}}\sqrt{\sigma_{22}}} + \frac{(x_{i,2} - \mu_2)^2}{(\sqrt{\sigma_{22}})^2} \right] = d_i^2$$

Additional term due to Covariance

Here $\rho_{12} = \frac{\sigma_{12}}{\sqrt{\sigma_{11}\sigma_{22}}}$ = Correlation Coefficient between $\mathbf{X}_1$ and $\mathbf{X}_2$

$\rho_{22} = 1$

$\rho_{11} = \frac{\text{Var}(X_1)}{\sqrt{\text{Var}(X_1)\text{Var}(X_1)}} = 1$
 $\rho_{12} = \frac{\text{Cov}(X_1, X_2)}{\sqrt{\text{Var}(X_1)\text{Var}(X_2)}}$

Chi-Square Distribution and Mahalanobis Distance

❖ What will be the shape of the following equation?

$$(\mathbf{x} - \boldsymbol{\mu})' \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) = d^2$$

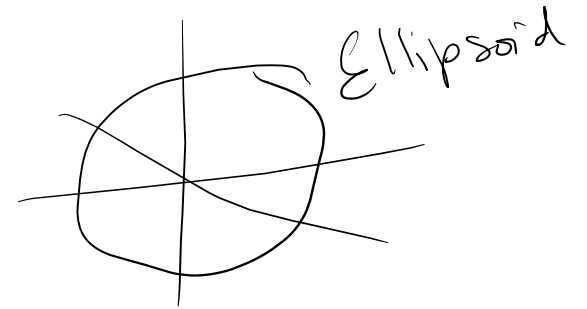
❖ Here $\mathbf{X}$ is p-variable vector ✓ *p features*

❖ For p- dimension, the shape will be an ellipsoid centered at $\boldsymbol{\mu} = [\mu_1, \mu_2, \dots, \mu_p]^T$.

❖ So for the most general case, we have

$$(\mathbf{x} - \boldsymbol{\mu})' \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) = d^2$$

which is an ellipsoid of p- axes



So How to Find d_i^2 ?

$$(\mathbf{x} - \boldsymbol{\mu})' \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) = d^2$$

The parameter d^2 is known as squared statistical distance of $\mathbf{x}$ from $\boldsymbol{\mu}$.

$$(\mathbf{x} - \boldsymbol{\mu})' \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) = d^2$$

follows chi- square (χ_p^2) distribution with p degrees of freedom

Centering
Scaling

$$\frac{X_i - \mu}{\sigma}$$

order of magnitudes are ~ equal

Standardizing

$$\frac{X_i - \mu}{\text{sigma}} = Z$$

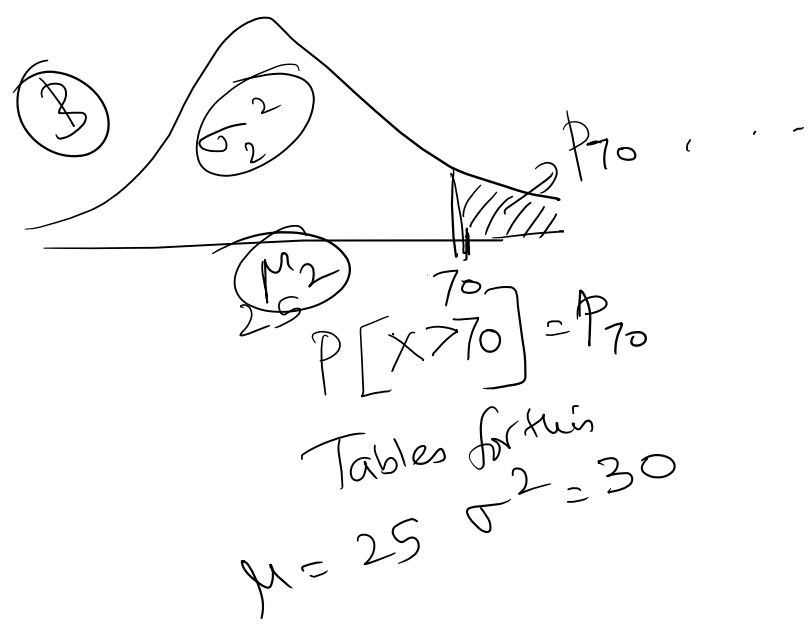
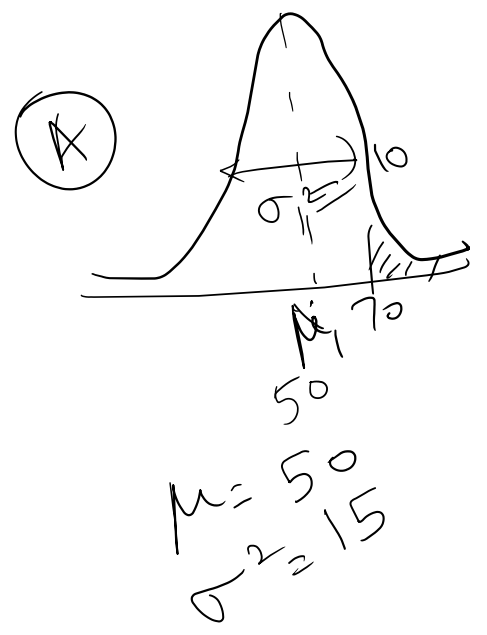
std. normal go for Standardizing.

$$E(Z) = \frac{E(X) - \mu}{\sigma}$$

$$Z = \frac{X - \mu}{\sigma} = \frac{\mu - \mu}{\sigma} = 0$$

Std. normal = variable

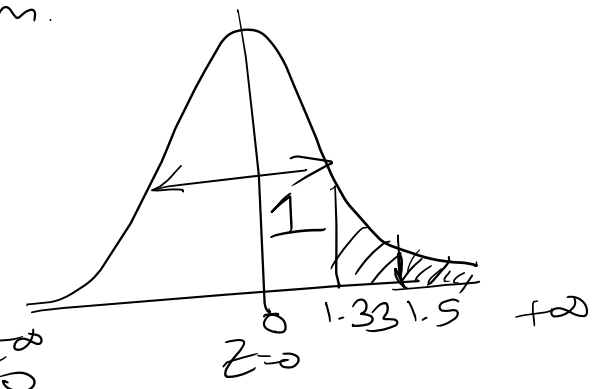
Mean of Z = 0 Std(Z) = 1



∞ distn.

$$Z_A = \frac{70 - 50}{15} = 1.333$$

$$Z_B = \frac{70 - 25}{30} = \frac{45}{30} = \frac{3}{2} = 1.5$$



Statistics

(Z) Normal DISTR.

analyzing means
of sample data

$$Z = \frac{X - \mu}{\sigma}$$

analyzing std dev/variance
of sample data

$$Z_1^2 = \left(\frac{x_1 - \mu_1}{\sigma_1} \right)^2$$

$$Z_2^2 = \left(\frac{x_2 - \mu_2}{\sigma_2} \right)^2$$

analyzing ratio of
variances of 2 sampled
data

$$Z_1^2 + Z_2^2 + \dots + Z_p^2$$

$$\sigma^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n-1}$$

Z

χ^2 DISTR.

F distr.

$$\frac{Z_1^2}{\left(\frac{x_1 - \mu_1}{\sigma_1} \right)^2} + \frac{Z_2^2}{\left(\frac{x_2 - \mu_2}{\sigma_2} \right)^2} + \dots + \frac{Z_p^2}{\left(\frac{x_p - \mu_p}{\sigma_p} \right)^2} \sim \chi_p^2$$

Chi-Square distr.
with p degrees of freedom

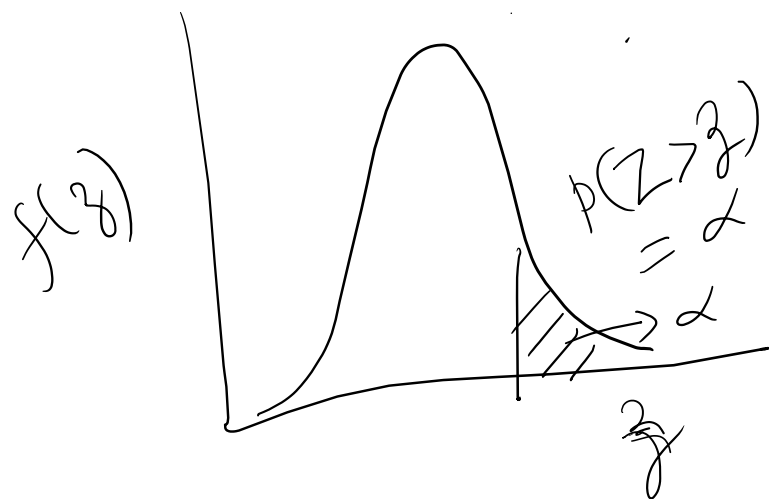
$$\frac{(x_1 - \mu_1)^2}{\sigma_1^2} + \frac{(x_2 - \mu_2)^2}{\sigma_2^2} + \dots + \frac{(x_p - \mu_p)^2}{\sigma_p^2}$$

$$\frac{Z_1^2}{(X_1 - \mu_1)^2} + \frac{Z_2^2}{(X_2 - \mu_2)^2} = d^2 \sim \chi_2^2$$

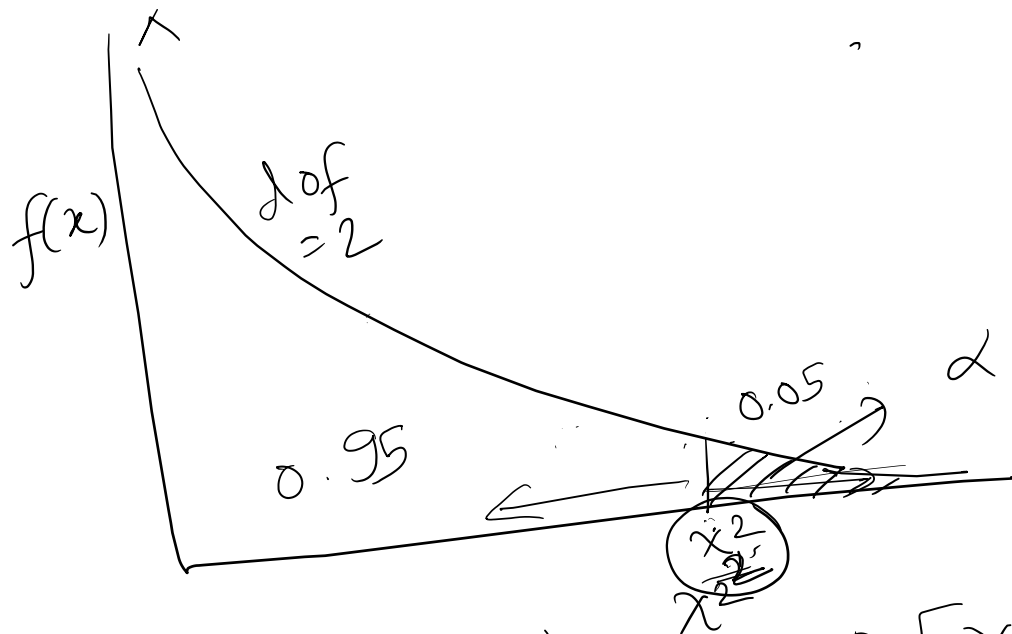
d^2 follows χ_2^2

chi square $\chi_2^2(\alpha)$
 $\alpha = 0.05$

$\alpha = 0.05$



Norm
 fresh



χ_2^2
 dis

$\chi_2^2 =$
 value

$$P[X^2 > \chi_2^2] = \alpha = 0.05$$

Chi-Square Distribution and Mahalanobis Distance

❖ The equation given below

$$(\mathbf{x} - \boldsymbol{\mu})' \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) = d^2$$

❖ Subscribes to the [chi-square distribution](#) with p-degrees of freedom.

❖ In other words, the [Mahalanobis distance](#) squared is the same as the chi-square variable value which may be associated with a probability or a fraction

$$(\mathbf{x} - \boldsymbol{\mu})' \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) \leq \chi_p^2(\alpha)$$

$$(\mathbf{x} - \bar{\mathbf{x}})' \mathbf{S}^{-1} (\mathbf{x} - \bar{\mathbf{x}}) \leq \chi_p^2(\alpha)$$

$$d^2 \leq \chi_p^2(\alpha)$$

Chi-Square Distribution and Mahalanobis Distance

$$(\mathbf{x} - \boldsymbol{\mu})' \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) \leq \chi_p^2(\alpha)$$

- ❖ **The Percentile Definition:** With respect to a chi-square distribution with p-degrees of freedom, the standard $100(1-\alpha)^{\text{th}}$ percentile means that a total area of $1-\alpha$ lies to the left (below) the $\chi_p^2(\alpha)$ value.
- ❖ The value of d^2 at $100(1 - \alpha)^{\text{th}}$ percentile value represents all multivariate- observations of $\mathbf{X}$ such that $100(1 - \alpha)\%$ lie below the $\chi_p^2(\alpha)$ value and each of these observations will have squared distances less than or equal to this value i.e., $d^2 \leq \chi_p^2(\alpha)$

$$(\mathbf{x} - \boldsymbol{\mu})' \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) \leq \chi_p^2(\alpha) \text{ with a probability of } 1-\alpha$$

Chi-Square Distribution and Mahalanobis Distance

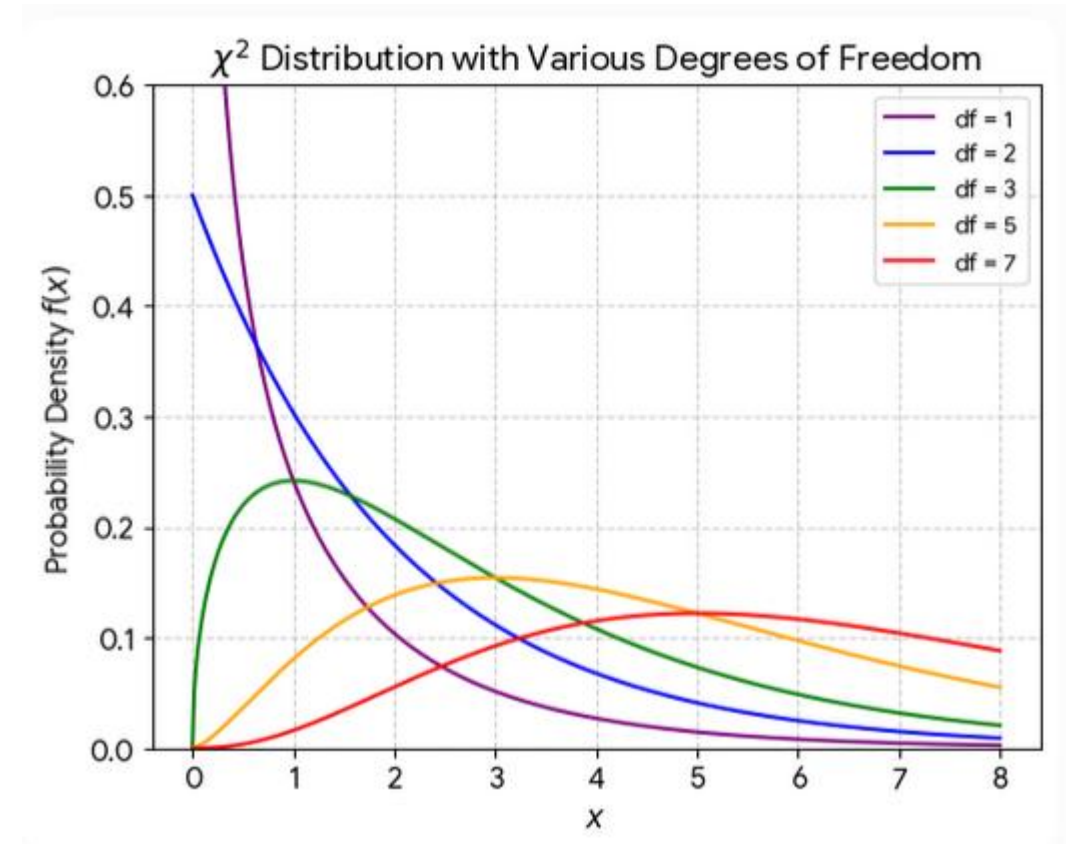
- ❖ Now we will see what d^2 is.
- ❖ d^2 is known as **squared statistical distance** of $\mathbf{x}$ from $\boldsymbol{\mu}$.
- ❖ The relation

$$(\mathbf{x} - \boldsymbol{\mu})' \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) \leq \chi_p^2(\alpha)$$

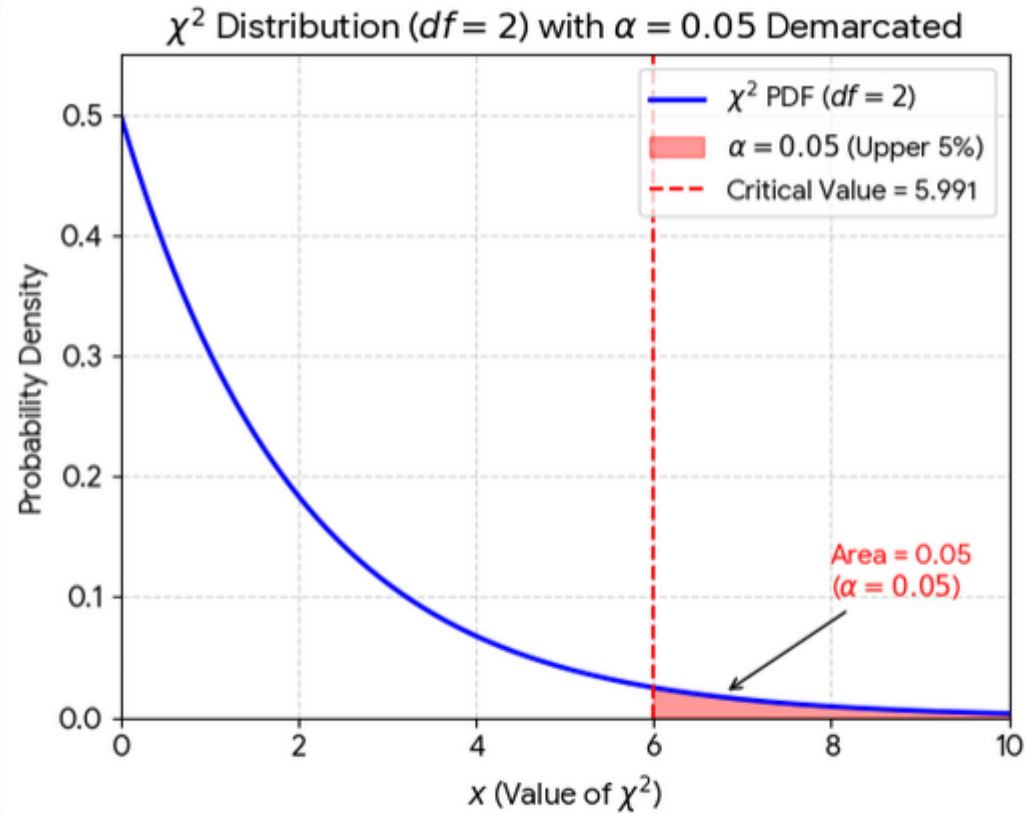
follows **chi- square χ^2 distribution with p degrees of freedom**

The pdf for χ^2 distribution is shown in Figure

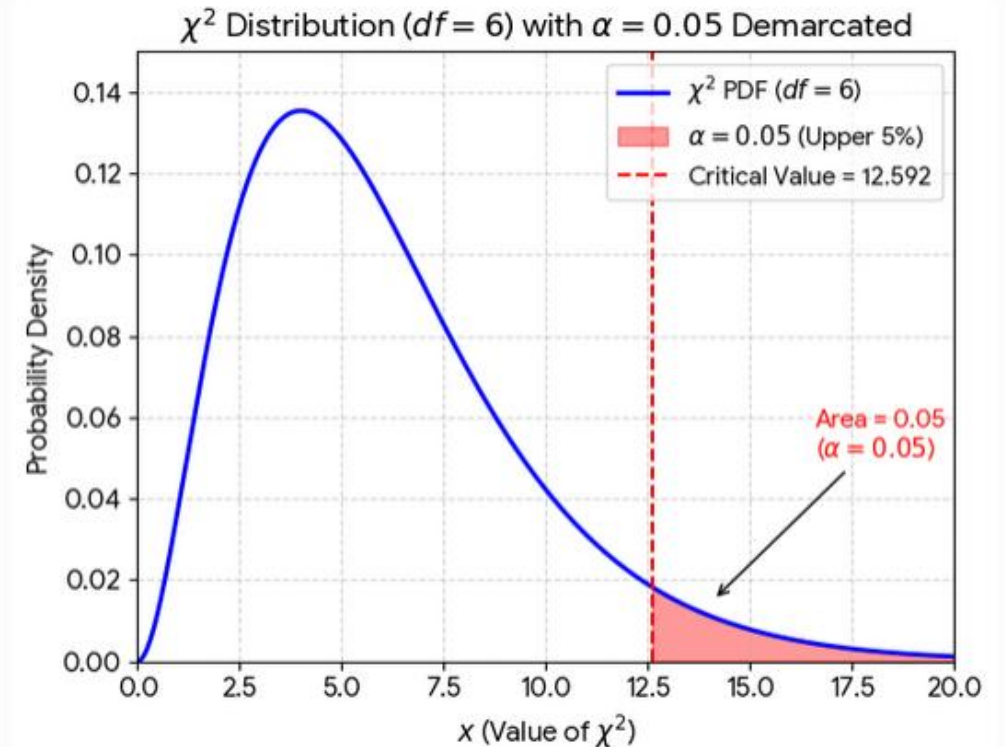
- ❖ Figure shows the pdf of χ^2 for 1, 2, 3, 5, and 7 degrees of freedom.



Chi-Square Distribution and Mahalanobis Distance



To demarcate the area corresponding to $\alpha = 0.05$ for a Chi-Squared (χ^2) distribution with 2 degrees of freedom ($df = 2$), we need to find the critical value where the upper-tail area equals exactly 0.05.

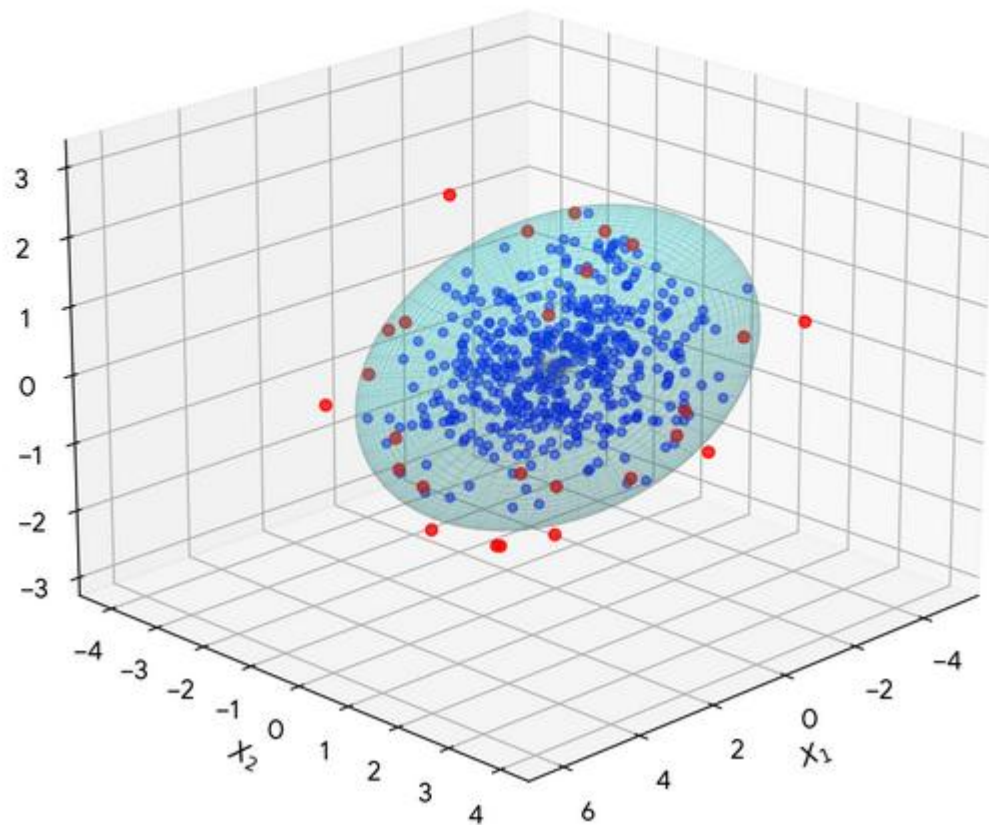


To demarcate the area corresponding to $\alpha = 0.05$ for a Chi-Squared (χ^2) distribution with 6 degrees of freedom ($df = 6$), we determine the critical threshold value that leaves exactly 5% of the total probability in the upper right tail.

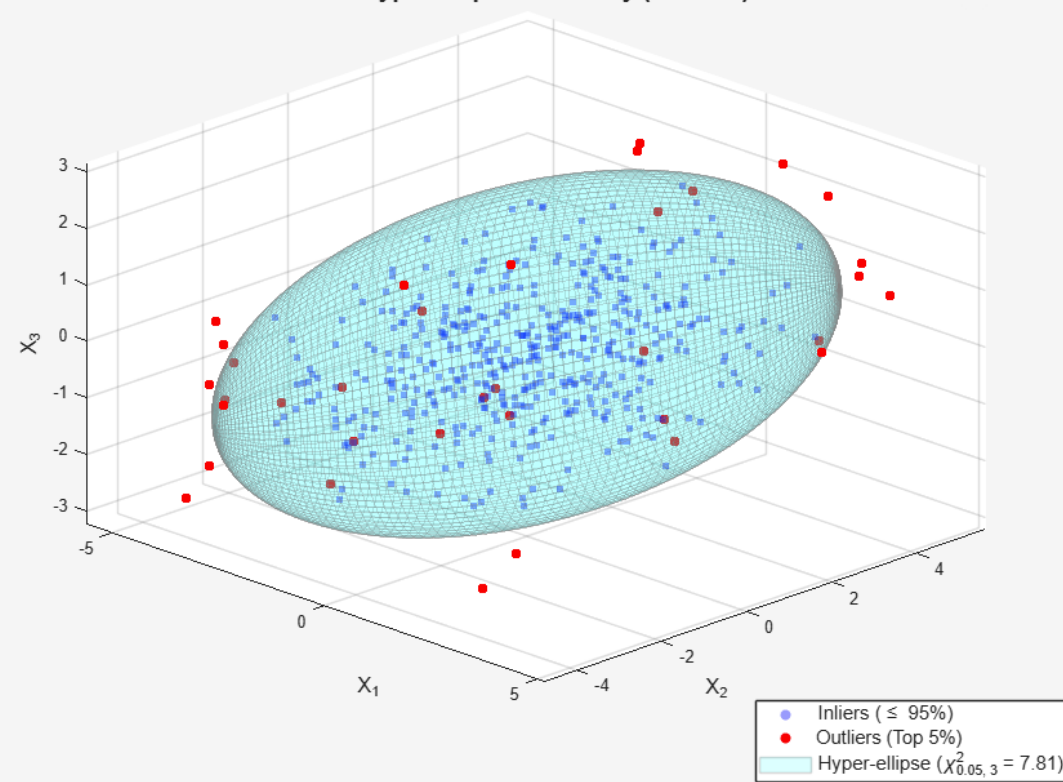
χ^2 Distribution & Mahalanobis Distance

Visualizing a 3D Hyper-ellipse (95% Confidence Ellipsoid)
Contours of Constant Mahalanobis Distance

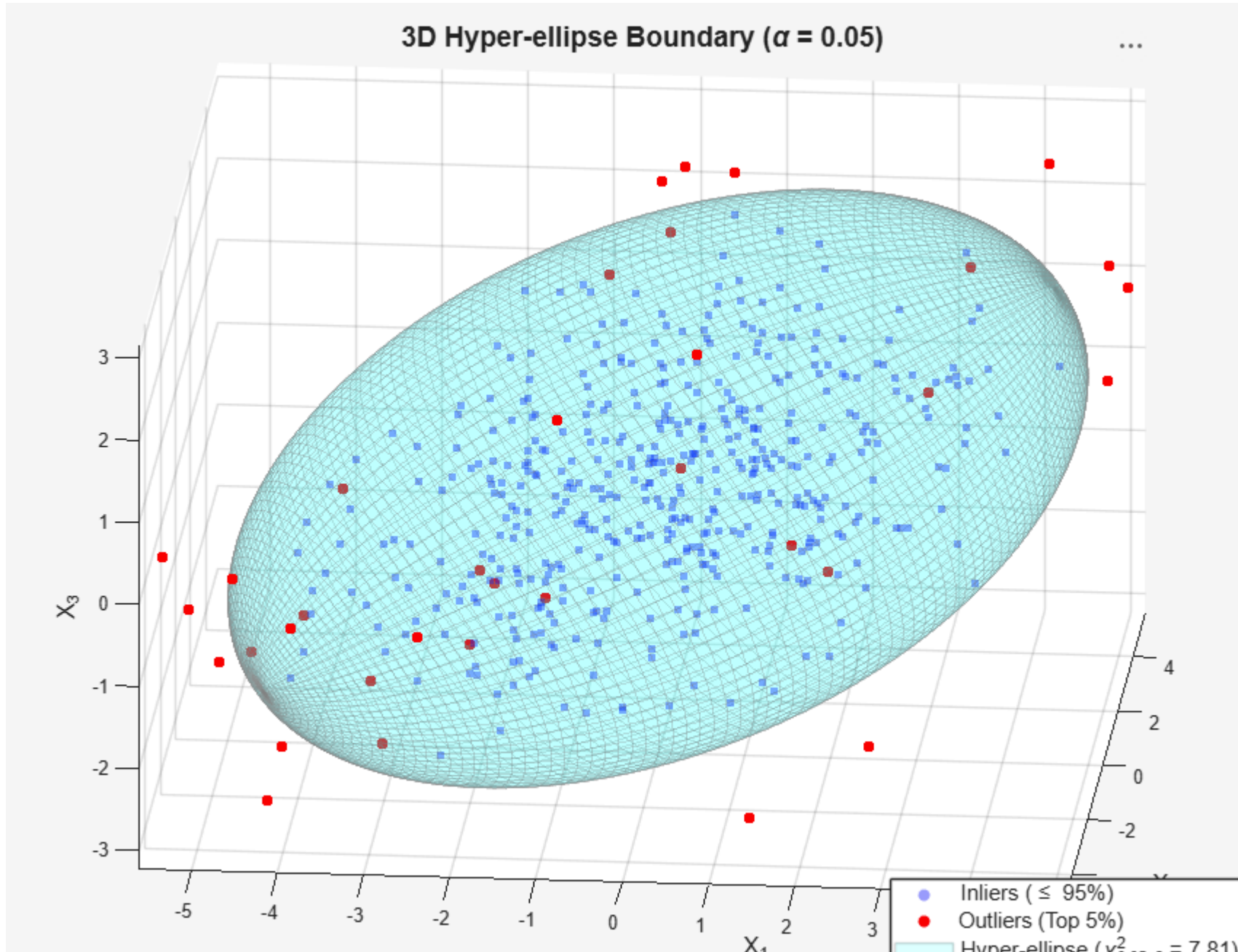
- Inliers (95%)
- Outliers (Top 5%)



3D Hyper-ellipse Boundary ($\alpha = 0.05$)



χ^2 Distribution & Mahalanobis Distance: Matlab



χ^2 Distribution & Mahalanobis Distance: Matlab Code

```
% MATLAB Script: Visualize a 3D Hyper-ellipse using Mahalanobis Distance
clear; clc; close all;

%% 1. Define Parameters and Generate Synthetic 3D Data
rng(42); % Set seed for reproducibility

% Define a mean vector and a positive-definite covariance matrix
mu = [0; 0; 0];
Sigma = [3, 1, 0.5;
         1, 2, 0.8;
         0.5, 0.8, 1];

% Generate 500 random samples from a Multivariate Normal Distribution
N = 500;
data = mvnrnd(mu, Sigma, N);

%% 2. Compute Chi-Squared Critical Value
p = 3; % Number of dimensions (degrees of freedom)
alpha = 0.05; % Significance level
% Critical value: 95% of data falls below this threshold
c2 = chi2inv(1 - alpha, p);

%% 3. Calculate Mahalanobis Distance to Identify Outliers
% Calculate squared Mahalanobis distance for each point manually or via mahal:
% d2 = mahal(data, mu, Sigma);
invSigma = inv(Sigma);
d2 = zeros(N, 1);
for i = 1:N
    diff = data(i, :) - mu;
    d2(i) = diff' * invSigma * diff;
end

% Separate data into inliers and outliers based on threshold
inliers = data(d2 <= c2, :);
outliers = data(d2 > c2, :);

%% 4. Generate the 3D Hyper-ellipse Geometry
% Get eigenvalues (lengths) and eigenvectors (directions) of Covariance Matrix
[V, D] = eig(Sigma);

% Generate a standard unit sphere mesh grid
[U, V_grid] = meshgrid(linspace(0, 2*pi, 100), linspace(0, pi, 100));
X_unit = cos(U) .* sin(V_grid);
Y_unit = sin(U) .* sin(V_grid);
Z_unit = cos(V_grid);
```

```
% Scale the sphere along the principal axes using eigenvalues and chi2 threshold
% Radii along principal axes are sqrt(lambda * c^2)
radii = sqrt(diag(D) * c2);
X_scaled = X_unit * radii(1);
Y_scaled = Y_unit * radii(2);
Z_scaled = Z_unit * radii(3);

% Rotate and translate the scaled ellipsoid using eigenvectors and the mean
ellipsoid_points = [X_scaled(:), Y_scaled(:), Z_scaled(:)] * V';
X_ellip = reshape(ellipsoid_points(:, 1), size(X_scaled)) + mu(1);
Y_ellip = reshape(ellipsoid_points(:, 2), size(Y_scaled)) + mu(2);
Z_ellip = reshape(ellipsoid_points(:, 3), size(Z_scaled)) + mu(3);

%% 5. Plotting the Results
figure('Position', [100, 100, 900, 700]);
hold on;

% Plot the inlier data points (Blue)
scatter3(inliers(:,1), inliers(:,2), inliers(:,3), 20, 'b', 'filled', ...
        'MarkerFaceAlpha', 0.4, 'DisplayName', 'Inliers (\leq 95%)');

% Plot the outlier data points (Red)
scatter3(outliers(:,1), outliers(:,2), outliers(:,3), 30, 'r', 'filled', ...
        'DisplayName', 'Outliers (Top 5%)');

% Render the hyper-ellipse surface mesh (Transparent Cyan)
surf(X_ellip, Y_ellip, Z_ellip, 'FaceColor', 'cyan', 'FaceAlpha', 0.15, ...
     'EdgeColor', [0.5 0.5 0.5], 'EdgeAlpha', 0.3, 'LineWidth', 0.5, ...
     'DisplayName', sprintf('Hyper-ellipse (\chi^2_{0.05, 3} = %.2f)', c2));

% Aesthetic configurations
title('3D Hyper-ellipse Boundary (\alpha = 0.05)', 'FontSize', 14);
xlabel('X_1', 'FontSize', 12);
ylabel('X_2', 'FontSize', 12);
zlabel('X_3', 'FontSize', 12);

grid on;
view(45, 20); % Set a clean isometric orientation angle
legend('Location', 'best', 'FontSize', 11);
axis equal; % Crucial for protecting true geometric aspect ratios
hold off;
```


χ^2 Distribution & Mahalanobis Distance: Python Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import chi2
import matplotlib.patches as mpatches

np.random.seed(42) # Set seed for reproducibility

# Define a mean vector and a positive-definite covariance matrix
mu = np.array([0, 0, 0])
Sigma = np.array([[3.0, 1.0, 0.5],
                  [1.0, 2.0, 0.8],
                  [0.5, 0.8, 1.0]])

# Generate 500 random samples from a Multivariate Normal Distribution
N = 500
data = np.random.multivariate_normal(mu, Sigma, N)
# Compute Chi-Squared Critical Value
p = 3 # Number of dimensions (degrees of freedom)
alpha = 0.05 # Significance level
# Critical value: 95% of data falls below this threshold
c2 = chi2.ppf(1 - alpha, p)
# Calculate Mahalanobis Distance to Identify Outliers
invSigma = np.linalg.inv(Sigma)
# Calculate squared Mahalanobis distance (Vectorized approach)
diff = data - mu
d2 = np.sum(np.dot(diff, invSigma) * diff, axis=1)
# Separate data into inliers and outliers based on threshold
inlier_mask = d2 <= c2
inliers = data[inlier_mask]
outliers = data[~inlier_mask]
# Generate the 3D Hyper-ellipse Geometry
# Get eigenvalues (lengths) and eigenvectors (directions) of Covariance Matrix
eigenvalues, eigenvectors = np.linalg.eigh(Sigma)

# Generate a standard unit sphere mesh grid
U = np.linspace(0, 2 * np.pi, 100)
V_grid = np.linspace(0, np.pi, 100)
U, V_grid = np.meshgrid(U, V_grid)
X_unit = np.cos(U) * np.sin(V_grid)
Y_unit = np.sin(U) * np.sin(V_grid)
Z_unit = np.cos(V_grid)
# Scale the sphere along the principal axes using eigenvalues and chi2 threshold
radii = np.sqrt(eigenvalues * c2)
X_scaled = X_unit * radii[0]
Y_scaled = Y_unit * radii[1]
Z_scaled = Z_unit * radii[2]
# Rotate and translate the scaled ellipsoid using eigenvectors and the mean
scaled_points = np.column_stack([X_scaled.ravel(), Y_scaled.ravel(), Z_scaled.ravel()])
ellipsoid_points = np.dot(scaled_points, eigenvectors.T)
```

```
X_ellip = ellipsoid_points[:, 0].reshape(X_scaled.shape) + mu[0]
Y_ellip = ellipsoid_points[:, 1].reshape(Y_scaled.shape) + mu[1]
Z_ellip = ellipsoid_points[:, 2].reshape(Z_scaled.shape) + mu[2] # Fixed index from [3] to [2]
# Plotting the Results
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
# Plot the inlier data points (Blue)
ax.scatter(inliers[:, 0], inliers[:, 1], inliers[:, 2],
           c='b', s=20, alpha=0.4, label='Inliers ( $\leq 95\%$ )')
# Plot the outlier data points (Red)
ax.scatter(outliers[:, 0], outliers[:, 1], outliers[:, 2],
           c='r', s=30, label='Outliers (Top 5%)')
# Render the hyper-ellipse surface mesh (Transparent Cyan)
ax.plot_surface(X_ellip, Y_ellip, Z_ellip, color='cyan', alpha=0.15,
               edgecolor='gray', linewidth=0.5)
# Added 'rf' (raw string) to fix the escape sequence error
surf_proxy = mpatches.Patch(color='cyan', alpha=0.3,
                             label=r'Hyper-ellipse ( $\chi^2_{\{0.05, 3\}} = \{c2:.2f\}$ )')
# Aesthetic configurations
ax.set_title(r'3D Hyper-ellipse Boundary ( $\alpha = 0.05$ )', fontsize=14)
ax.set_xlabel('$X_1$', fontsize=12)
ax.set_ylabel('$X_2$', fontsize=12)
ax.set_zlabel('$X_3$', fontsize=12)
# Set a clean isometric orientation angle (elev, azim)
ax.view_init(elev=20, azim=45)
# Assemble legend
handles, labels = ax.get_legend_handles_labels()
handles.append(surf_proxy)
ax.legend(handles=handles, loc='best', fontsize=11)
# Aspect ratio lock for true geometric geometry
max_range = np.array([X_ellip.max()-X_ellip.min(),
                      Y_ellip.max()-Y_ellip.min(),
                      Z_ellip.max()-Z_ellip.min()]).max() / 2.0
mid_x = (X_ellip.max()+X_ellip.min()) * 0.5
mid_y = (Y_ellip.max()+Y_ellip.min()) * 0.5
mid_z = (Z_ellip.max()+Z_ellip.min()) * 0.5

ax.set_xlim(mid_x - max_range, mid_x + max_range)
ax.set_ylim(mid_y - max_range, mid_y + max_range)
ax.set_zlim(mid_z - max_range, mid_z + max_range)
ax.set_box_aspect([1, 1, 1])

plt.show()
```

Multivariate Normal Distribution

- ❖ For the normal (Gaussian) distribution, we may determine whether an observation is likely to belong to a predefined population, depending upon the number of standard deviations it is away from the mean.
- ❖ If the probability is 5%, then we only expect to find an observation that far from the mean with a probability of 1 in 20
- ❖ Likewise, for a multinormal distribution, the distance from the mean or center of the measurements (represented by the means μ_1 and μ_2) may be used to **determine the likelihood of membership** of a predefined population.

Multivariate Normal Distribution

- ❖ Finding distance in more than one dimension does not have a forward (+) or backward (-) sign,

The statement means that when you calculate the separation between points in spaces with two or more dimensions (like (2D) or (3D) coordinate planes), the result is an absolute, directionless scalar. Unlike 1D movement (where (+) and (-) represent "forward" and "backward"), multi-dimensional distance has no predefined positive or negative sign

- ❖ So, we use squared distances
- ❖ Will the squared distance from the mean of a dataset tell us how likely an observation is part of the crowd i.e., a predefined population, analogous to the single variable case?
- ❖ The rest of the points without membership to the population are classified as outliers

Chi-Square Distribution

- ❖ The square of each individual standardized normal variable must fall into an underlying chi square distribution.
- ❖ So, must $Z_1^2 + Z_2^2 + \cdots + Z_p^2 = \chi_p^2$.
- ❖ Note that there is no multiplier constant before these standard normal variables Z s.

Mahalanobis Distance

- ❖ So, how can we use chi square distribution to determine whether an observation belongs to a predefined population by looking at the distance from the centre $(0,0)$ or (μ_1, μ_2) ?
- ❖ When only one variable is measured, it is of course simple
- ❖ A new measurement of distance comes into play, the Mahalanobis distance.

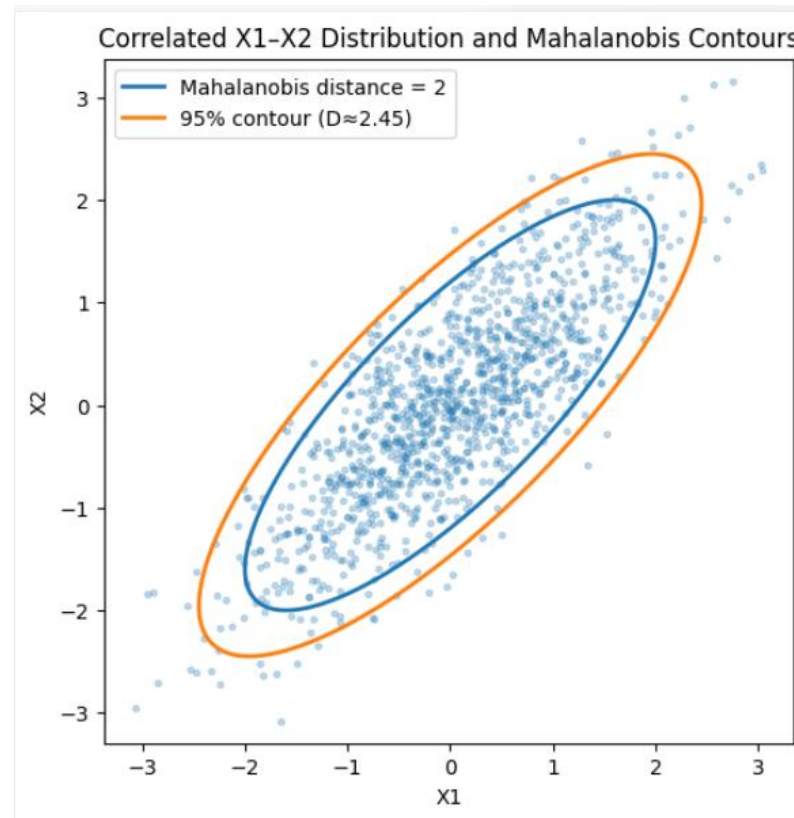
Mahalanobis Distance

- ❖ Normal standardization (Z-score normalization) places correlated variables on the same scale by transforming them to have a mean of 0 and a standard deviation of 1
- ❖ This removes the influence of different units (e.g., kg, temperature) and magnitudes (0-1000 kg versus -5 to +100°C, allowing the variables to contribute equally to analysis based on their **relative deviation from the mean**, not their raw units.

$$(\mathbf{x} - \boldsymbol{\mu})' \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) \leq \chi_p^2(\alpha)$$

Mahalanobis Distance

- ❖ Even with 2 variables with correlation, the above expression leads to the equation of an ellipse whose principal axes are orthogonal
- ❖ Figure below is of the Mahalanobis distance of 2 (or a squared distance of 4) units from the centre of a bivariate normal distribution.



Mahalanobis Distance

- ❖ Figure in previous slide is of the Mahalanobis distance of 2 (or a squared distance of 4) units from the centre of a **bivariate normal** distribution.
- ❖ If an underlying distribution is multinormal, we expect the Mahalanobis distances to be characterized by a chi square distribution.
- ❖ The number of degrees of freedom of the chi-square distribution equals the number of variables.
- ❖ So, for two variables, it has 2 degrees of freedom

Mahalanobis Distance

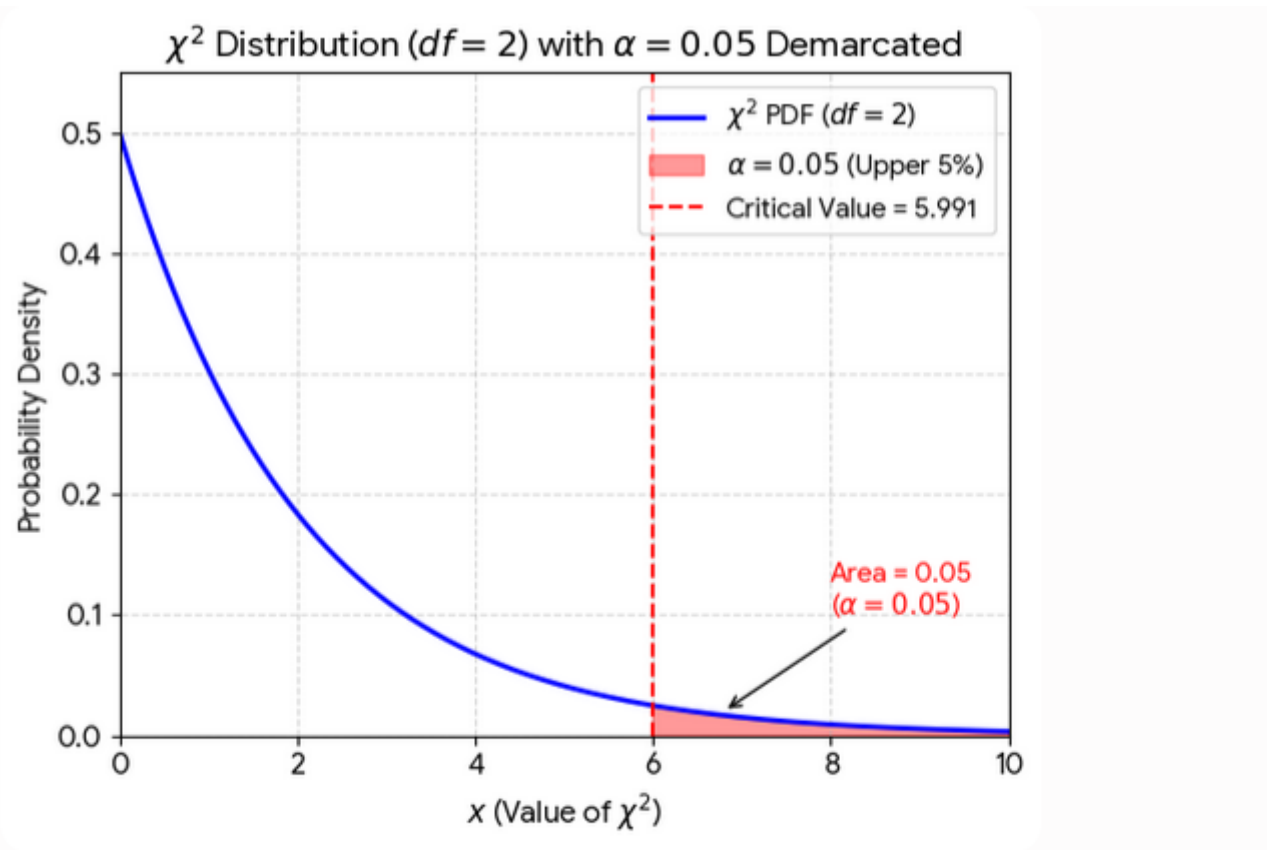
- ❖ Using Excel, we predict the # of observations that is outside a defined number of Mahalanobis distances from the centre.
- ❖ The percentage predicted to be further than 2 Mahalanobis distance units from the centre is given by `CHISQ.DIST.RT(4,2)`, where 4 is the square of 2 (the number of units, remember this is squared) and 2 is the number of degrees of freedom.

Mahalanobis Distance

- ❖ Chi-Square distribution is only adhered to only if the underlying distribution is multinormal
- ❖ So, one could calculate the squared Mahalanobis distances of samples that are not multinormal, and they would NOT adhere to a chi square distribution
- ❖ Let us consider the upper $100(1 - \alpha)^{\text{th}}$ percentile of the d^2 distribution.

Mahalanobis Distance

- ❖ The value of d^2 at the upper $100(1 - \alpha)^{\text{th}}$ percentile value represents all multivariate-observations of $\mathbf{X}$ such that $(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) \leq \chi_p^2(\alpha)$ with a probability $1 - \alpha$.
- ❖ When $\alpha = 0.05$, we get a d^2 value at 95th percentile point (see Figure in next slide) which is $\chi_p^2(0.05)$
- ❖ Once the value of d^2 is determined, use Equation $(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) = d^2$ to construct the ellipsoid.
- ❖ When $p = 2$, the ellipsoid becomes an ellipse.



To demarcate the area corresponding to $\alpha = 0.05$ for a Chi-Squared (χ^2) distribution with 2 degrees of freedom ($df = 2$), we need to find the critical value where the upper-tail area equals exactly 0.05.

Code links

[2D and 3D Scatterplot code \(both Python and Matlab\)](#)

[Centering scatterplot \(both Python and Matlab\)](#)

[Mahalanobis Distance \(both Python and Matlab\)](#)